

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université Benyoucef BENKHEDDA-Alger1



Faculté des Sciences
Département des Mathématiques et Informatique

Projet de Fin d'Etudes pour l'obtention du diplôme de
Master en Informatique

Option : Réseaux et Systèmes Embarqués

Thème

Étude et mise en place d'un système de
stockage réparti Ceph

Encadré par :

- M^r Abbas Amine
- M^r Neffah Mohamed

Réalisé par :

- TATA Melissa
- LEBCIR Souheib El Mehdi

2021/2022

Remerciements

Avant de présenter le rapport on voudrait remercier toutes les personnes ayant contribué à enrichir nos connaissances dont nos professeurs de la faculté d'Alger. On remercie particulièrement nos encadrants qui nous ont suivi tout au long de ce projet jusqu'à son aboutissement.

On remercie aussi nos proches qui ont cru en nous et qui ont mis à notre disposition tous les moyens nécessaires et possibles pour arriver là où nous sommes aujourd'hui.

Dédicace

On dédie ce projet

À nos frères et sœurs qui ont cru en nous,

À nos grands-parents qui attendent notre réussite,

À nos parents qui ont contribué à la réussite de ce projet grâce à leur appui et leur encouragement.

Que ce travail soit l'accomplissement de leurs vœux tant allégués et le fruit de leur soutien infaillible.

TATA Melissa et LEB CIR Souheib EL Mehdi

Résumé

Etant stagiaires à l'entreprise nationale Sonatrach, notre objectif fut de mettre en place une solution de stockage distribué Ceph, un possible alternatif à leur système actuel SAN et NAS. Conçu pour répondre aux besoins de stockage de données, elle pallie aux problèmes de flexibilité, d'élasticité, de faible tolérance aux pannes et de coût élevé. Cette dernière fournit aussi des interfaces pour plusieurs types de stockage au sein d'un seul cluster réduisant ainsi les frais généraux. Tout ceci fait de Ceph une parfaite sélection pour être la nouvelle plateforme de stockage pour les nombreuses données importantes de Sonatrach.

Afin d'atteindre le but de ce projet, nous sommes passés par plusieurs étapes : Familiarisation avec Ceph, sa mise en place et sa conception à grande échelle.

Mots clés : Système distribué - Ceph - Cluster - SAN - NAS - Software defined storage.

Abstract

As trainees at the national company Sonatrach, our objective was to set up a Ceph distributed storage solution, a possible alternative to their current system SAN and NAS. Designed to meet data storage needs, it overcomes the problems of flexibility, elasticity, low fault tolerance and high cost. It also provides interfaces for multiple storage types within a single cluster, reducing overhead. All this makes Ceph a perfect choice to be the new storage platform for Sonatrach's many important data.

In order to achieve the goal of this project, we went through several steps: Familiarization with Ceph, its implementation and large-scale design.

Keywords: Distributed system - Ceph - Cluster - SAN - NAS - Software defined storage.

ملخص

بصفتنا متدربين في شركة سوناطراك الوطنية، كان هدفنا إنشاء حل تخزين موزع Ceph، كبديل محتمل لنظامهم الحالي SAN وNAS. تم تصميمه لتلبية احتياجات تخزين البيانات، فهو يتغلب على مشاكل المرونة، والتسامح المنخفض مع الخطأ والتكلفة العالية. كما يوفر واجهات لأنواع تخزين متعددة داخل مجموعة واحدة، مما يقلل من الحمل. كل هذا يجعل Ceph خيارًا مثاليًا ليكون منصة التخزين الجديدة للعديد من بيانات سوناطراك المهمة.

من أجل تحقيق هدف هذا المشروع، مررنا بعدة مراحل: التعرف على Ceph، تنفيذه وتصميمه على نطاق واسع.

الكلمات المفتاحية: النظام الموزع - Ceph - Cluster - NAS - SAN - التخزين المعرف بالبرمجيات.

Tables des matières

Tables des matières	I
Liste des figures.....	III
Liste des abréviations.....	V
Introduction générale	1
Contexte général.....	1
Problématique et motivation.....	1
Contribution	2
Plan du mémoire	3
Chapitre 1 : Présentation de l'entreprise d'accueil	4
1.1 Introduction.....	4
1.2 Sonatrach.....	4
1.3 Historique de l'entreprise.....	5
1.4 Logo de l'entreprise	6
1.5 Situation géographique.....	6
1.6 Organigramme de l'entreprise	6
1.7 Les activités de l'entreprise.....	7
1.8 Produits de l'entreprise	7
1.8.1 Les hydrocarbures liquides :	7
1.8.2 Les hydrocarbures gazeux :	7
1.9 Les systèmes de stockage qu'utilise Sonatrach.....	7
1.9.1 Étude de l'existant.....	7
a) Description de l'existant	7
b) Critique de l'existant.....	9
1.9.2 Solution envisagée	10
1.9.3 Avantages qu'a Ceph sur le SAN et le NAS.....	10
a) En termes de prix	10
b) En termes d'agilité.....	11
c) En termes d'évolutivité	11
1.10 Conclusion	11
Chapitre 2 : Étude de la solution Ceph	12
2.1 Introduction.....	12
2.2 Notion de base	12
2.2.1 Système distribué	12
2.2.2 Cluster de stockage	13
2.2.3 Software-Defined Storage.....	14
2.3 Présentation de Ceph.....	14

2.4 Architecture de Ceph.....	15
2.4.1 RADOS	15
2.4.2 CRUSH	16
2.4.3 Nœuds et démons Ceph.....	17
2.5 Le stockage des données dans le cluster Ceph.....	18
2.6 La gestion des objets au niveau de l'infrastructure	18
2.7 Les modes de protection des données.....	19
2.8 Les caractéristiques de Ceph	20
2.9 Conclusion	21
<i>Chapitre 3 : Mise en place d'un cluster Ceph.....</i>	<i>22</i>
3.1 Introduction.....	22
3.2 Environnement de travail.....	22
3.2.1 Architecture du cluster	22
3.2.2 Matériels utilisés	23
3.2.3 Logiciels et outils utilisés.....	24
3.3 La réalisation.....	25
3.3.1 Virtualisation du server physique : Installation de KVM	25
3.3.2 Déploiement de l'environnement Ceph cluster.....	25
a) Création du réseau public et cluster	25
b) Déploiement des nœuds templates.....	28
c) Clonage des nœuds pour obtenir 3 OSD, 3 MON et un client.....	30
d) Attachement des 6 disques virtuels aux nœuds OSDs	33
3.3.3 Installation des prérequis de Ceph	37
a) Dans le serveur.....	37
b) Dans le mon1	37
c) Dans l'osd1	40
3.3.4 Installation de Ceph avec l'outil Cephadm	44
3.3.5 Création de la Crush Map et stockage des données	51
3.4 Conception à grande échelle.....	56
3.4.1 Architecture à grande échelle.....	56
3.4.2 Spécification des besoins	58
3.5 Conclusion	58
<i>Conclusion générale</i>	<i>59</i>
<i>Netographie</i>	<i>60</i>

Liste des figures

Figure 1.1 : Logo de Sonatrach.....	6
Figure 1.2 : Organigramme de la macrostructure de Sonatrach.....	6
Figure 1.3 : Architecture de NAS.....	8
Figure 1.4 : Architecture de SAN.....	9
Figure 2.1 : Illustration d'un système distribué.....	12
Figure 2.2 : Illustration d'un cluster de stockage.....	13
Figure 2.3 : Logo de Ceph.....	14
Figure 2.4 : Architecture de Ceph.....	15
Figure 2.5 : Illustration d'une Crush map.....	17
Figure 2.6 : Stockage des objets.....	18
Figure 2.7 : Gestion des objets.....	19
Figure 2.8 : Réplication des objets.....	19
Figure 2.9 : Codage d'effacement.....	20
Figure 3.1 : Architecture du cluster Ceph.....	22
Figure 3.2 : Logo « KVM ».....	24
Figure 3.3 : Logo « PuTTY ».....	24
Figure 3.4 : Initialisation du réseau public.....	26
Figure 3.5 : Création du réseau public.....	26
Figure 3.6 : Activation du démarrage automatique du réseau public.....	26
Figure 3.7 : Démarrage du réseau public.....	26
Figure 3.8 : Initialisation du réseau cluster.....	27
Figure 3.9 : Création du réseau cluster.....	27
Figure 3.10 : Activation du démarrage automatique du réseau cluster.....	27
Figure 3.11 : Démarrage du réseau cluster.....	28
Figure 3.12 : Liste des réseaux virtuels libvirt actifs.....	28
Figure 3.13 : Affichage du fichier téléchargé.....	28
Figure 3.14 : Affichage de l'état du serveur Apache.....	29
Figure 3.15 : Récupération des informations du site http.....	30
Figure 3.16 : Liste de toutes les machines actives et inactives.....	31
Figure 3.17 : Arrêt du serveur centos8tempmon.....	32
Figure 3.18 : Arrêt du serveur tempOSd.....	32
Figure 3.19 : Clonage des machines virtuelles.....	33
Figure 3.20 : Démarrage des machines virtuelles.....	35
Figure 3.21 : Adresses IP des machines virtuelles.....	36
Figure 3.22 : Connection SSH au mon1.....	37
Figure 3.23 : Arrêter et désactiver Firewallld.....	37
Figure 3.24 : Installation du paquet yum-utils.....	38
Figure 3.25 : Configuration du repository.....	38
Figure 3.26 : Installation de Docker.....	39
Figure 3.27 : Démarrage et activation de Docker.....	39
Figure 3.28 : Installation de Python3.....	39
Figure 3.29 : Configuration de NTP.....	40
Figure 3.30 : Affichage des sources NTP.....	40
Figure 3.31 : Connexion SSH à l'osdl.....	40
Figure 3.32 : Arrêter et désactiver Firewallld.....	41

Figure 3.33 :	Installation du paquet yum-utils.....	41
Figure 3.34 :	Configuration du repository.....	41
Figure 3.35 :	Installation de Docker.....	42
Figure 3.36 :	Démarrage et activation de Docker.....	42
Figure 3.37 :	Configuration de NTP pour l'osd1.....	43
Figure 3.38 :	Affichage des sources NTP.....	43
Figure 3.39 :	Interface graphique de Ceph.....	45
Figure 3.40 :	Etat du cluster Ceph.....	46
Figure 3.41 :	Liste des hôtes sur l'invite de commande.....	46
Figure 3.42 :	Liste des hôtes sur l'interface graphique.....	46
Figure 3.43 :	Ajout des hôtes au cluster de stockage.....	47
Figure 3.44 :	Liste des hôtes après leur ajout.....	48
Figure 3.45 :	Liste des hôtes.....	48
Figure 3.46 :	Liste des appareils disponibles pour déployer les OSDs.....	48
Figure 3.47 :	Consommation de tout périphérique de stockage disponible et inutilisé.....	49
Figure 3.48 :	Liste des OSD sur l'interface graphique.....	49
Figure 3.49 :	Déploiement des moniteurs.....	49
Figure 3.50 :	Liste des MON sur l'interface graphique.....	50
Figure 3.51 :	Etat du cluster après sa mise en place.....	50
Figure 3.52 :	Liste des hôtes et leurs rôles.....	50
Figure 3.53 :	Illustration d'une Datacenter.....	51
Figure 3.54 :	Carte CRUSH.....	53
Figure 3.55 :	Ajout du pool « pooldata01 ».....	53
Figure 3.56 :	Création de la règle CRUSH et l'assigner au pool.....	54
Figure 3.57 :	Information sur le cluster.....	54
Figure 3.58 :	Liste des objets dans le pool « pooldata01 ».....	54
Figure 3.59 :	Emplacement de l'objet créé.....	55
Figure 3.60 :	Distribution des clusters Ceph au niveau national.....	56
Figure 3.61 :	Architecture du déploiement de Ceph.....	57
Figure 3.62 :	Architecture des clusters Ceph.....	57

Liste des abréviations

CephFS : Ceph File System.

CIFS : Common Internet File System.

CRUSH : Controlled Replication Under Scalable Hashing.

FCP : Fibre Channel Protocol.

FUSE : Filesystem in Userspace.

HTTP : HyperText Transfer Protocol.

iSCSI : Internet Small Computer System Interface.

KVM : Kernel-based Virtual Machine.

MDS : Ceph Metadata Server.

MGR : Ceph Manager daemon.

MON : Ceph Monitor daemon.

NAS : Network Attached Storage.

NAT : Network Address Translation.

NFS : Network File System.

NTP : Network Time Protocol.

OSD : Object Storage Daemon.

PG : Placement Group.

POSIX : Portable Operating System Interface uniX.

RADOS : Reliable Autonomic Distributed Object Store.

RBD : Rados Bloc Device.

RGW : Rados Gateway.

SAN : Storage Area Network.

SDS : Software Defined Storage

SSH : Secure Shell.

VM : Virtual Machine.

Introduction générale

Contexte général

Sonatrach est une compagnie nationale algérienne d'envergure internationale. Elle intervient dans l'exploitation, le transport par canalisations, la transformation et la commercialisation des hydrocarbures et de leurs dérivées. Sa mission est de valoriser de façon optimale les ressources nationales d'hydrocarbures et de créer des richesses au service du développement économique et social du pays.

Cette société cherche continuellement de nouvelles technologies afin d'améliorer son fonctionnement. Cette fois-ci, c'est vers une nouvelle solution de stockage qu'elle souhaiterait se tourner. Pour cela, le centre d'innovation du département informatique a décidé d'évaluer une nouvelle solution plus adaptée aux besoins de l'entreprise en termes de coûts et de performance.

Problématique et motivation

Notre projet est motivé par le besoin de Sonatrach de trouver une nouvelle solution de stockage fiable, autonome et évolutive pour ses nombreuses données sensibles. Ce besoin est le résultat de plusieurs inconvénients présents dans leur système de stockage actuel, citons les points importants :

- ❖ Extension difficile.
- ❖ Coût élevé.
- ❖ Manque d'autonomie : Elle dépend toujours du prestataire.

Ayant trouvé du potentiel dans la solution Ceph, Sonatrach nous a confié la tâche d'effectuer une étude approfondie sur Ceph afin de leur fournir des informations qui pourraient éventuellement les aider dans leurs démarches de recherche d'une meilleure solution de stockage.

Contribution

Nous avons étudié et mis en place une solution de stockage distribué nommé Ceph permettant de :

- ❖ Réduire les coûts de déploiement et de maintenance.
- ❖ Assurer la sécurité des données.
- ❖ Assurer la disponibilité des données.
- ❖ Augmenter la tolérance aux pannes.
- ❖ Rendre le système flexible et élastique.
- ❖ Fournir un environnement autonome et autoréparant.
- ❖ Faire face à l'explosion des données.

Pour réaliser notre projet nous avons dû :

- ❖ Se familiariser avec :
 - Les systèmes de stockage distribués.
 - Ceph et son fonctionnement.
 - Les différents réseaux de stockage.
- ❖ Chercher comment les entreprises stockaient leurs données.
- ❖ Comprendre l'architecture de stockage utilisée au sein de la Sonatrach.
- ❖ Etudier la faisabilité de la solution Ceph au niveau de la Sonatrach.
- ❖ Comprendre l'architecture de Ceph.
- ❖ Apprendre à déployer un cluster Ceph.
- ❖ Choisir les meilleurs outils pour déployer la solution Ceph.
- ❖ Mettre en place un cluster Ceph sur un serveur sous CentOS8 Stream.
- ❖ Illustrer notre solution sur un cas concret.

Plan du mémoire

Nous avons organisé notre rapport autour de trois chapitres :

- ❖ Le premier chapitre est consacré à :
 - La présentation de l'organisme d'accueil « SONATRACH ».
 - L'analyse et la spécification des besoins de l'entreprise pour cerner la problématique et mieux comprendre la solution.
- ❖ Le deuxième chapitre permet de se familiariser avec le système de stockage distribué Ceph.
- ❖ Le troisième chapitre est consacré à :
 - La mise en place du cluster Ceph.
 - L'illustration d'un déploiement à grande échelle de la solution Ceph pour le réseau de raffinerie de pétrole de Sonatrach.

Chapitre 1 : Présentation de l'entreprise d'accueil

1.1 Introduction

Sonatrach fait partie de ces géants qui tendent à dépasser leurs limites et à augmenter leur productivité. Dans ce but, la société cherche régulièrement à moderniser ses infrastructures et elle n'hésite pas à aller chercher plus loin pour s'adapter à ses nouveaux besoins et trouver ce qu'il lui correspond le mieux à un coût optimal. Aujourd'hui, cette dernière veut se tourner vers une autre solution de stockage qui serait plus adéquate à son fonctionnement.

Ce chapitre visera à présenter Sonatrach dans un premier lieu puis passer à l'analyse de ses besoins, à la critique de l'existant et à l'évaluation de l'éventuelle solution dans un second lieu.

1.2 Sonatrach

SONATRACH, acronyme de « Société Nationale de Transport et Commercialisation des Hydrocarbures », est une compagnie étatique algérienne et un acteur international majeur dans l'industrie des hydrocarbures. Elle est classée 1ère en Afrique et 12ème dans le monde en 2021, toutes activités confondues, avec un chiffre d'affaires à l'exportation de plus de 35 milliards de dollar.

Née le 31 décembre 1963, la compagnie intervient dans l'exploration, la production, le transport par canalisations, la transformation et la commercialisation des hydrocarbures et de leurs dérivés. Elle est 4ème exportateur mondial de GNL, 3ème exportateur mondial de GPL et 5ème exportateur de Gaz Naturel.

Adoptant une stratégie de diversification, Sonatrach se développe aussi bien dans les activités de génération électrique, d'énergies nouvelles et renouvelables, de dessalement d'eau de mer, de recherche et d'exploitation minière. Poursuivant sa stratégie d'internationalisation, Sonatrach opère en Algérie et dans plusieurs régions du monde. [\[1\]](#)

1.3 Historique de l'entreprise

Après l'indépendance, l'Etat Algérien a pris la décision de s'approprier ses richesses pétrolières et gazières, et de se doter d'un instrument de développement réunissant toutes les conditions de sa souveraineté, par la création de la Sonatrach. [\[2\]](#)

Le 31 décembre 1963, Sonatrach est créée. Elle est placée sous tutelle du ministère de l'énergie et des industries pétrochimiques, et régie par le droit commercial Algérien.

En 1965, la Sonatrach pose le premier pipeline Algérien reliant le champ de Haoud El Hamar- Arzew d'un diamètre de 28 pouces et d'une longueur de 801 Km.

Le 22 Septembre 1966, le gouvernement définit la nouvelle mission de Sonatrach d'une manière à élargir son domaine d'activité en industrie pétrolière, à savoir la recherche, la production et la transformation de plus de la commercialisation et le transport des hydrocarbures, en prenant des participations dans les concessions détenues par certaines sociétés étrangères telle que : ESSO, SHELL, SINCLAIR, MOBIL ELF...etc [\[2\]](#)

Le 24 février 1971, La Sonatrach a connu la plus grande et la plus importante transformation de son histoire, elle s'est vue confier la tâche de gérer et développer toutes les branches de l'industrie pétrolière et gazière algérienne après que le gouvernement ait décidé de nationaliser les hydrocarbures. [\[2\]](#)

En 1980, la Sonatrach gère 10% du pétrole et du gaz algériens.

En 1981, Dix-sept nouvelles entreprises nationales ont vu le jour, dont : ASMIDAL, ANIP, EMPC, ENAFOR, NAFTAL, NAFTEC, KNPC...etc.

En 1998, la Sonatrach devient une société par actions (SPA).

En 2001, la Sonatrach s'est vu définir le schéma d'organisation de sa macro structure qui a connu plusieurs réaménagements par la suite.

Aujourd'hui la Sonatrach est l'acteur majeur de l'industrie pétrolière et gazière au service de l'économie algérienne. Elle intervient également dans d'autres secteurs tels que la génération électrique, les énergies nouvelles et renouvelables et le dessalement d'eau de mer. Elle exerce ses activités en Algérie et partout dans le monde où des opportunités se présentent.

Avec plus de 200 000 travailleurs, Sonatrach compte à ce jour 154 filiales : L'exploitation, le raffinage, la commercialisation, le stockage, les services aux puits, etc. [2]

1.4 Logo de l'entreprise



Figure 1.1 : Logo de Sonatrach

1.5 Situation géographique

Le siège social de la direction générale du groupe SONATRACH est sis à Djenane El Malik – Hydra – Alger.

1.6 Organigramme de l'entreprise

Le schéma d'organisation de la macrostructure de Sonatrach est représenté dans la **Figure 1.2** ci-dessous.

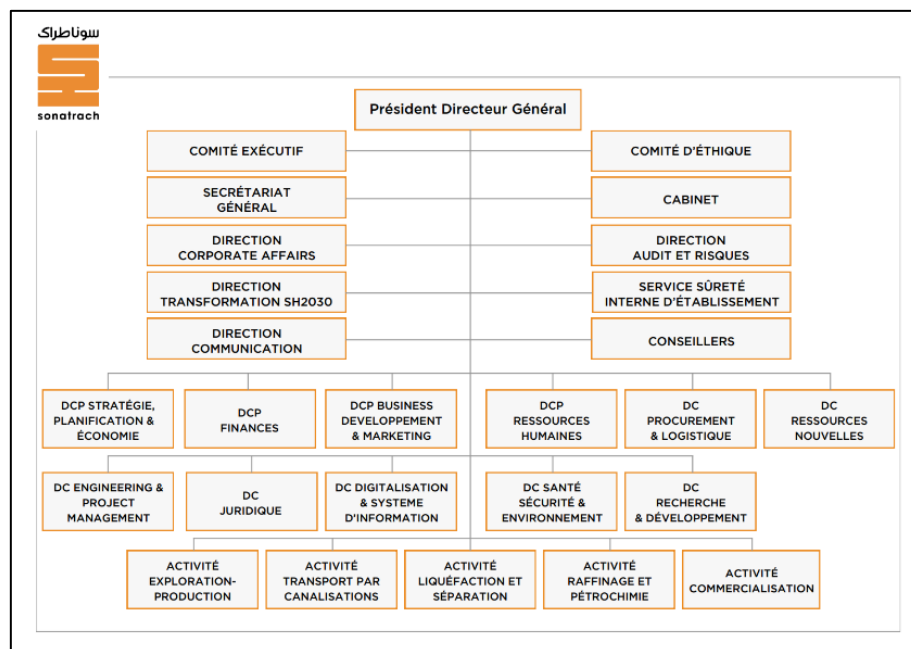


Figure 1.2 : Organigramme de la macrostructure de Sonatrach

1.7 Les activités de l'entreprise

SONATRACH couvre l'ensemble de la chaîne de valeur des hydrocarbures. Ainsi ses activités dans leur globalité incluent :

- Exploration et Production.
- Transport Par Canalisations.
- Liquéfaction et Séparation.
- Raffinage et Pétrochimie.
- Commercialisation.

1.8 Produits de l'entreprise

La gamme de produits de SONATRACH se compose de deux parties :

1.8.1 Les hydrocarbures liquides :

- Le Gaz du Pétrole Liquéfié (GPL).
- Le Pétrole brut.
- Le Condensât.
- Les produits raffinés : essences, naphta, jetkero, gas-oil et fuel.

1.8.2 Les hydrocarbures gazeux :

- Le Gaz Naturel.
- Le Gaz Naturel Liquéfié (GNL).

1.9 Les systèmes de stockage qu'utilise Sonatrach

1.9.1 Étude de l'existant

a) Description de l'existant

Aujourd'hui, les entreprises possèdent différentes méthodes pour stocker leurs données. En fonction du type de la donnée, de son importance et de sa confidentialité. Les manières dont les entreprises stockent ces données diffèrent grandement. Jusqu'à présent, les entreprises utilisent des supports de stockage tels que le NAS, le SAN et le stockage objet. D'autres les externalisent sur le Cloud. Sonatrach utilise les techniques de stockage en réseau SAN et NAS.

❖ Le stockage NAS :

Le NAS (Network Attached Storage) est un appareil de stockage autonome qui se présente sous la forme d'une boîte rectangulaire prête à recevoir un ou plusieurs disques durs qui va fonctionner sur le réseau local par le biais d'une connexion Ethernet, voire en wifi. Il héberge les fonctions « serveur » de protocoles de partage de fichiers qui sont implantés dans le NAS tels que CIFS (Common Internet File System, associé à Microsoft) ou NFS (Network File System, associé à Linux). Il permet ainsi de fournir un espace de stockage important à des clients divers qui le verront ainsi comme un espace de stockage distant. Les équipements NAS, très proches des serveurs de fichiers dans le principe, présentent les mêmes avantages. En outre, ces machines spécialisées, affichent d'excellentes performances, une sécurité accrue et une souplesse d'administration. Son architecture se présente comme dans la **Figure 1.3.** [\[15\]](#)

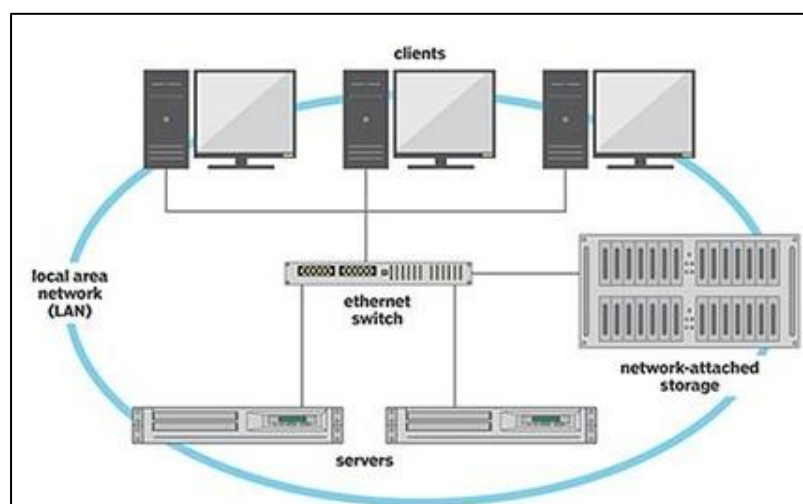


Figure 1.3 : Architecture de NAS

❖ Le stockage SAN :

Le SAN (Storage Area Network) est un réseau haut débit dédié et indépendant qui interconnecte plusieurs serveurs et leur offre des pools partagés de périphériques de stockage. Chaque serveur peut accéder à un stockage partagé comme s'il s'agissait d'un lecteur y étant directement rattaché. Un réseau SAN est généralement constitué de câbles, d'adaptateurs de bus hôte et de commutateurs SAN reliés à des baies de stockage et à des serveurs. Chaque commutateur et système de stockage du réseau de stockage doit être interconnecté. Les protocoles actuellement les plus utilisés pour la communication au sein d'un SAN sont : FCP (Fibre Channel Protocol) et iSCSI (Internet Small Computer System Interface). Son architecture se présente comme dans la **Figure 1.4.** [\[14\]](#)

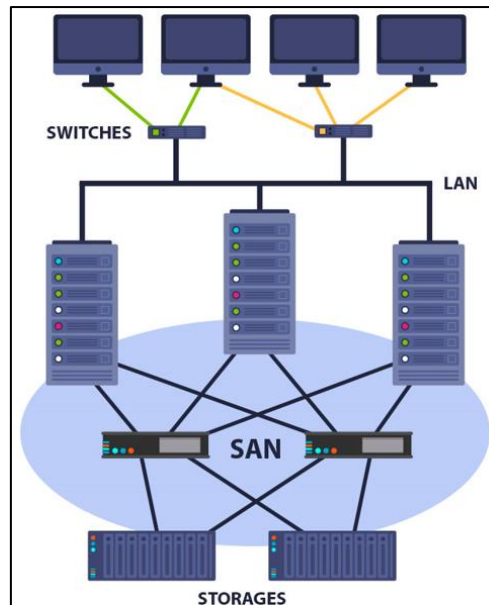


Figure 1.4 : Architecture de SAN

b) Critique de l'existant

Les systèmes existants présentent des points négatifs :

- ❖ Configuration complexe : Les réseaux SAN nécessitent souvent des outils de gestion et un bon niveau d'expertise.
- ❖ Le réseau SAN présente des problèmes d'interopérabilité.
- ❖ Le matériel nécessaire pour le SAN est très cher : Si l'entreprise souhaite élargir sa capacité de stockage, elle devra déboursier des sommes colossales.
- ❖ La gestion du SAN est difficile, l'entreprise doit constamment avoir du personnel qualifié dans ses rangs.
- ❖ Manque de flexibilité : Les modifications dans l'installation SAN prennent beaucoup de temps.
- ❖ Le stockage NAS ne peut envoyer que des fichiers, et non des blocs de données.
- ❖ Le fait que NAS utilise le réseau IP de l'entreprise, peut engendrer des congestions de ce dernier, particulièrement pour de gros transferts de données.
- ❖ NAS n'est pas pris en charge par toutes les applications.
- ❖ Avec le NAS, toute contraction dans le réseau local peut ralentir le temps d'accès au stockage.

1.9.2 Solution envisagée

Afin de réduire leurs coûts de stockage, mais aussi leur dépendance vis-à-vis des grands fournisseurs de systèmes de stockage traditionnels, de plus en plus d'entreprises sont tentées d'utiliser les solutions de Software-Defined Storage libres, notamment lorsqu'elles déploient des technologies de cloud comme OpenStack ou des technologies d'analyse Big Data comme Hadoop/Spark. L'une des technologies de stockage open source les plus prometteuses et les plus populaires est Ceph.

Ceph est un système de stockage distribué qui a la particularité de délivrer à la fois des services de stockage en mode bloc, en mode objet et en mode fichier. Ceph est notamment devenu populaire dans le monde du stockage du fait de ses nombreux points d'intégration avec OpenStack qui en font la technologie de stockage la plus déployée avec le framework de cloud libre.

Ceph réplique les données et les rend tolérantes aux pannes, en utilisant du matériel de base, Ethernet IP et ne nécessitant aucun support matériel spécifique. Le système de Ceph offre une reprise après sinistre et une redondance des données grâce à des techniques telles que la réplication et le codage d'effacement qu'on verra dans le prochain chapitre. En raison de sa conception, le système est à la fois auto-réparateur et auto-géré, visant à minimiser le temps d'administration et les autres coûts. Ce qu'offre Ceph comme avantage correspond aux besoins de nombreuses entreprises dont Sonatrach.

1.9.3 Avantages qu'a Ceph sur le SAN et le NAS

a) En termes de prix

La solution de stockage Ceph est plus abordable que les SAN et NAS traditionnels, en grande partie parce qu'il ne repose pas sur un ensemble de matériel coûteux pour assurer la fiabilité.

Généralement, le stockage des données est coûteux ; les fournisseurs de SAN et de NAS ont toujours dépendu de matérielles et de systèmes d'exploitation sur mesure, qui peuvent coûter entre 10 et 20 millions de dollars en coûts de développement.

À l'inverse, Ceph exploite le matériel x86 et les systèmes d'exploitation standards. Il n'y a pas de pièces personnalisées coûteuses. Au lieu de dépendre d'un matériel incroyablement coûteux, Ceph dépend que du logiciel. [\[3\]](#)

b) En termes d'agilité

L'agilité est la vitesse à laquelle le fournisseur de stockage peut apporter des modifications. Dans le cas des fournisseurs traditionnels de NAS et de SAN, les changements prennent des semaines, voire des mois. Quant à la solution Ceph, les mêmes modifications peuvent se faire instantanément. [\[3\]](#)

c) En termes d'évolutivité

Les baies SAN ou NAS traditionnelles sont extrêmement limitées dans leur évolutivité. Même les baies SAN conçues pour être évolutives ne peuvent évoluer que jusqu'à environ 16 contrôleurs, et ce uniquement si elles n'utilisent pas de SSD. Ceph n'est pas limité de cette manière, il peut évoluer jusqu'à des centaines de milliers de nœuds, ce qui signifie qu'il est infiniment évolutif. [\[3\]](#)

1.10 Conclusion

Dans ce chapitre, nous avons commencé par présenter Sonatrach. Ensuite, nous avons tenté de mettre en évidence les points qui la pousseraient à changer le système de stockage. En contrepartie, nous avons tenté de mettre en lumière la solution Ceph. Dans le prochain chapitre nous ferons une étude détaillée sur Ceph pour mieux comprendre son fonctionnement.

Chapitre 2 : Étude de la solution Ceph

2.1 Introduction

Ceph est un logiciel hautement fiable, facile à gérer et gratuit. Il offre une évolutivité extraordinaire : des milliers de clients accèdent à des exaoctets de données. Ceph garantit une grande flexibilité dans le choix des appareils de stockage et une importante modularité.

Pour mieux se familiariser avec ce dernier, on lui a consacré tout ce chapitre où on l'a présenté ainsi que son fonctionnement avant d'entamer sa mise en place.

2.2 Notion de base

2.2.1 Système distribué

Un système distribué, comme illustré dans la **Figure 2.1**, est un environnement informatique dans lequel divers composants sont répartis sur plusieurs ordinateurs ou autres dispositifs informatique appartenant à un même réseau. Ces appareils divisent le travail et coordonnent leurs efforts pour effectuer des tâches plus efficacement qu'un seul appareil. [4]

Cette architecture très flexible est reconnue pour sa disponibilité, car en cas de panne l'architecture ne s'effondre pas et seul le service touché est indisponible contrairement à l'architecture centralisé. [5]

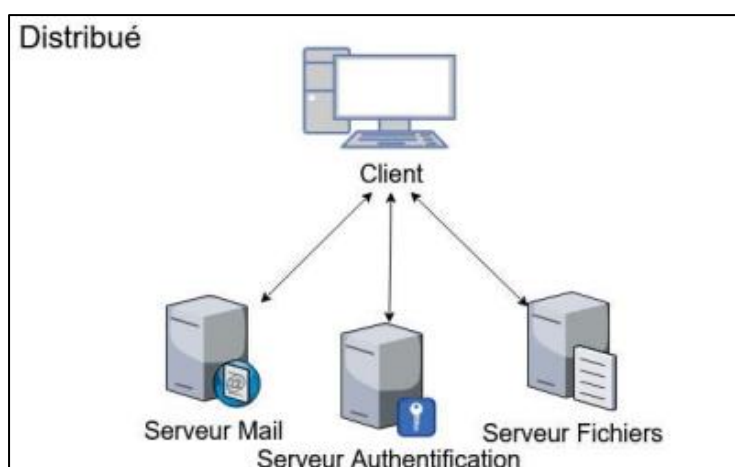


Figure 2.1 : Illustration d'un système distribué

2.2.2 Cluster de stockage

Le cluster ou grappe de serveurs, est une approche d'architecture distribuée dans laquelle un ensemble d'ordinateurs étroitement connectés travaillent ensemble et sont vus comme un seul ordinateur par l'utilisateur. Les ordinateurs qui constituent le cluster portent le nom de nœuds. Les nœuds sont regroupés dans des casiers ou racks. Les nœuds d'un casier sont connectés entre eux à travers la fibre optique ou des réseaux LAN à très haut débit, de l'ordre du Gigaoctet à la seconde (voir **Figure2.2**). Le cluster peut être composé de plusieurs casiers de nœuds, dans ce cas les casiers sont connectés entre eux soit par un autre niveau de câble réseau ou au moyen de commutateurs. Pour étendre le cluster il suffit simplement d'ajouter de nouveaux nœuds. Il n'y'a théoriquement pas de limites au nombre de nœuds qui peuvent être ajoutées au cluster. Les activités des nœuds du cluster sont planifiées et gérées par un orchestrateur de tâches. [6]

Dans Ceph, le cluster de stockage est la base de tous les déploiements. Basé sur un logiciel nommé RADOS (Reliable Autonomic Distributed Object Store), il se compose de plusieurs types de démons : un démon de stockage, de surveillance et de gestion. Un cluster de stockage Ceph peut contenir des milliers de nœuds de stockage. Un système optimal possède au moins un démon de surveillance et deux démons de stockage.

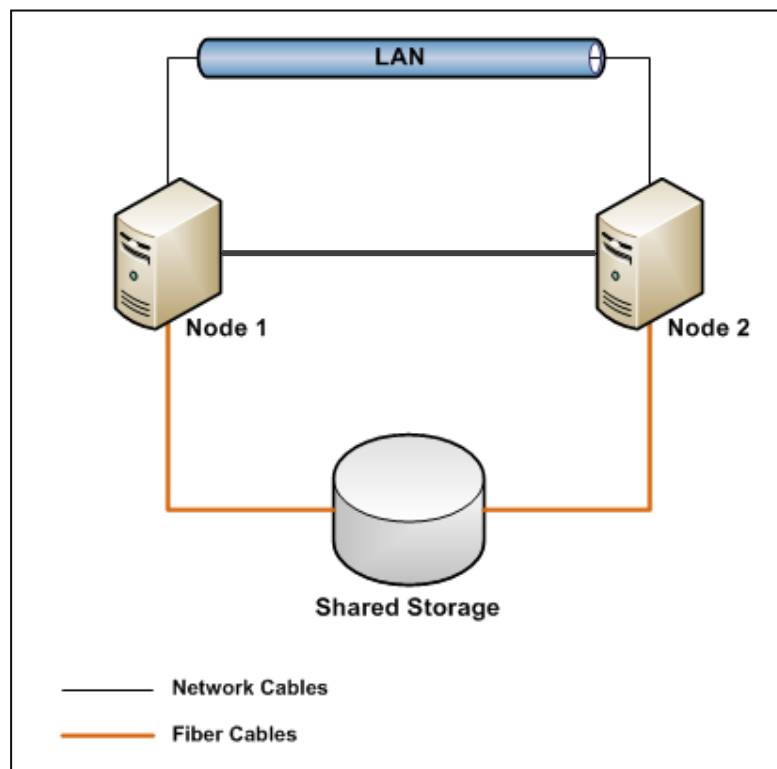


Figure 2.2 : Illustration d'un cluster de stockage

2.2.3 Software-Defined Storage

Le software-defined storage ou le stockage défini par logiciel, est un terme utilisé en informatique pour désigner le stockage de données basé sur des solutions logicielles. [7]

Le stockage défini par logiciel apporte une abstraction entre le logiciel et le matériel au sein d'une solution de stockage. Il s'oppose aux formes de stockage telles que les NAS ou les SAN qui dépendent de matériels et logiciels indissociables. [7]

L'un des intérêts du software-defined storage est de bénéficier d'une architecture de stockage basée sur un ensemble de logiciels, agnostiques des technologies matérielles, et qui permettrait d'automatiser le provisionnement de stockage au sein d'une entreprise. [7]

2.3 Présentation de Ceph

Ceph est une solution libre de stockage distribué (software-defined storage) très populaire qui propose trois protocoles en un avec : Bloc, Fichiers et Objet. Les objectifs principaux de Ceph sont d'être complètement distribués sans point unique de défaillance, extensible jusqu'à l'exaoctet et librement disponible. Les données sont répliquées, permettant au système d'être tolérant aux pannes. [8]

Ceph fonctionne sur du matériel standard (serveurs x86) dit commoditaire et de l'Ethernet IP. [8]

Le système est conçu pour s'autoréparer et automatiser au maximum ses tâches administratives afin de réduire les coûts d'exploitation, et d'être agnostique sur le choix du Hardware associé (disques, serveurs commoditaires). [8]

Ceph est souvent utilisé par les centres de données qui ont adopté l'Open source comme leur nouvelle norme pour le stockage blocs à forte croissance, le stockage objet et le Big data. [8]



Figure 2.3 : Logo de Ceph

2.4 Architecture de Ceph

Ceph se base sur une architecture Client/Cluster. Cette architecture est distribuée et modulaire. Elle permet de mettre en œuvre des systèmes de stockage couvrant un large spectre de besoin. Selon le type de serveur mis en œuvre, la nature du stockage utilisé et la performance des interfaces réseaux, il est possible de bâtir des configurations répondant à des contraintes différentes. On peut ainsi créer des clusters optimisés. De manière générale l'architecture de Ceph se présente comme dans la **Figure 2.4** ci-dessous.

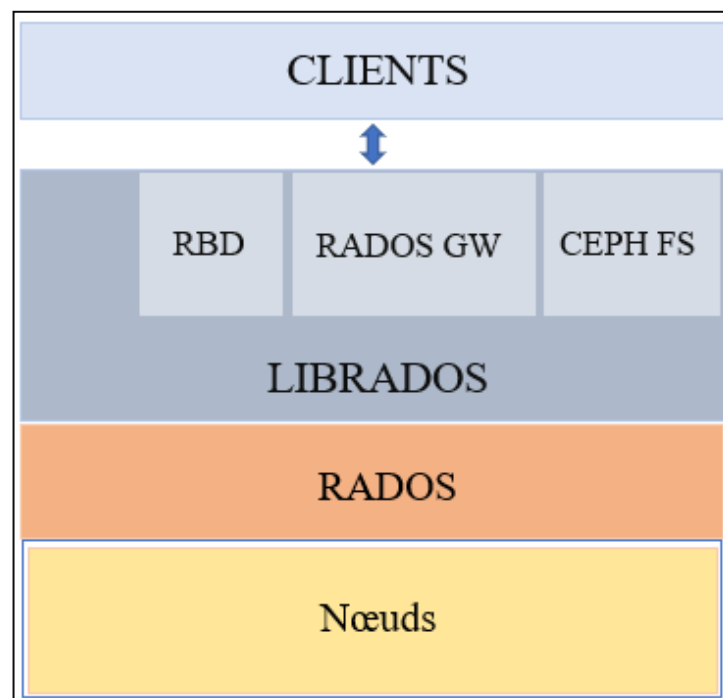


Figure 2.4 : Architecture de Ceph

2.4.1 RADOS

Le composant de base de Ceph est appelé RADOS (Reliable Autonomic Distributed Object Store). Il est chargé de gérer les données stockées dans le cluster. Les données de Ceph sont généralement stockées en tant qu'objets. Chaque objet se compose d'un identificateur et des données.

RADOS fournit les méthodes d'accès suivantes aux objets stockés qui couvrent de nombreux cas d'utilisation :

- ❖ **Librados** : est une librairie native C qui permet un accès direct à RADOS. Elle est utilisée pour les accès bloc, objet et système de fichier. Les applications peuvent l'implémenter.

- ❖ **Radosgw** : RGW ou RADOS Gateway est une interface de stockage d'objets conçue librados, elle permet de lire et écrire des données à l'aide du protocole Internet HTTP. RGW prend en charge deux interfaces : Compatible S3 et compatible avec Swift.
- ❖ **CephFS** : Ceph Filesystem ou CephFS est un système de fichiers compatible POSIX qui utilise un cluster Ceph pour stocker les données. Pour utiliser cette fonctionnalité, il est nécessaire d'avoir une structure Ceph Metadata Server (MDS) dans le cluster Ceph.
- ❖ **RADOS Block Device** : RBD ou RADOS Block Device est un logiciel qui facilite le stockage de données basées sur des blocs dans le Ceph open source.

2.4.2 CRUSH

Au cœur d'un cluster Ceph se trouve l'algorithme CRUSH. CRUSH est l'acronyme de Controlled Replication Under Scalable Hashing. C'est une fonction qui gère l'allocation de stockage et nécessite relativement peu de paramètres. Les paramètres sont une carte actuelle du cluster, comprenant le statut du cluster, certaines règles de placement définies par l'administrateur et le nom de l'objet qui doit être stocké ou récupéré. Avec ces informations, tous les nœuds de Ceph sont en mesure de calculer l'emplacement auquel un objet et ses répliques sont stockés. Cela rend l'écriture ou la lecture des données très efficace. CRUSH tente de distribuer équitablement les données sur tous les nœuds de la grappe. [\[9\]](#)

La carte CRUSH contient tous les nœuds de stockage et les règles de placement définies par l'administrateur pour le stockage des objets dans la grappe. Elle définit une structure hiérarchique qui correspond généralement à la structure physique de la grappe. Cette structure peut être utilisée pour définir des domaines de défaillance. Ceph s'assure ensuite que les répliques sont stockées sur différentes branches d'un domaine de défaillance spécifique. [\[9\]](#)

Si le domaine de défaillance est défini sur le niveau rack, les répliques d'objets sont réparties sur différents racks. Cela peut limiter les interruptions de service provoquées par un commutateur défaillant sur un rack. Si une unité de distribution de l'alimentation dessert une rangée de racks, le domaine de défaillance peut être défini sur rangée. En cas de défaillance de l'unité de distribution de l'alimentation, les données répliquées sont toujours disponibles sur les autres rangées. (Voir **Figure2.5**) [\[9\]](#)

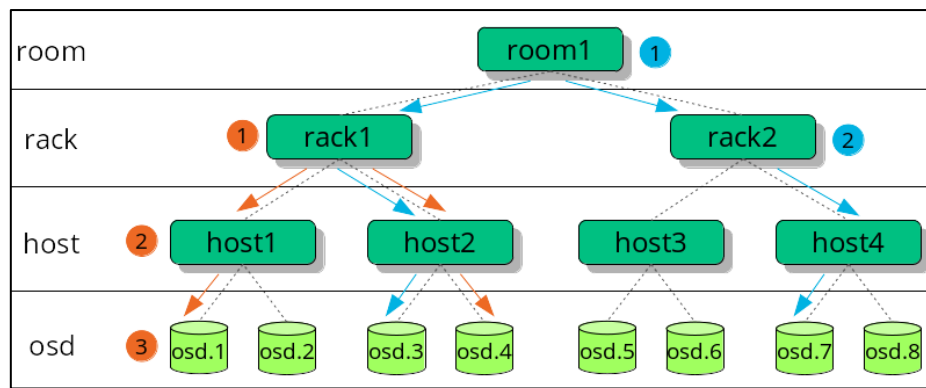


Figure 2.5 : Illustration d'une Crush map

2.4.3 Nœuds et démons Ceph

Dans Ceph, les nœuds sont des serveurs fonctionnant pour la grappe. Ils peuvent exécuter divers types de démons. Il est recommandé de n'exécuter qu'un seul type de démon sur chaque nœud, à l'exception des démons manager (MGR) qui peuvent être colocalisés avec des démons moniteurs (MON). Chaque grappe nécessite au moins les démons MON, MGR et OSD.

- ❖ **Ceph Monitor** : Les nœuds Ceph Monitor (MON) gèrent les informations sur le statut de la grappe, la carte de tous les nœuds et les règles de distribution des données. En cas de défaillance ou de conflit, les nœuds Ceph Monitor de la grappe décident, à la majorité, des informations qui sont correctes. Pour former une majorité admissible, il est recommandé de disposer d'un nombre impair de nœuds Ceph Monitor au minimum trois.
- ❖ **Ceph Manager** : Ceph Manager (MGR) collecte les informations d'état de l'ensemble de la grappe. Le démon Ceph Manager s'exécute parallèlement aux démons Ceph Monitor. Il fournit une surveillance supplémentaire et assure l'interface avec les systèmes de surveillance et de gestion externes.
- ❖ **Ceph OSD** : Ceph OSD ou Ceph Object Storage Devices est un démon qui gère des périphériques de stockage d'objets qui sont des unités de stockage physiques ou logiques. Les périphériques de stockage d'objets peuvent être des disques/partitions physiques ou des volumes logiques. Le démon prend également en charge la réplication et le rééquilibrage des données en cas d'ajout ou de suppression de nœuds. Les daemons Ceph OSD communiquent avec les daemons Ceph Monitor et leur fournissent l'état des démons OSD voisins. Si le moniteur Ceph ne reçoit pas de rapports ou s'il reçoit des rapports de modifications dans le cluster de stockage Ceph, le moniteur Ceph met à jour l'état de la carte du cluster Ceph.

- ❖ **Ceph MDS** : Ceph MDS est le démon du serveur de métadonnées pour le système de fichiers distribué Ceph. Il enregistre les chemins de stockage, le nom des fichiers et l'horodatage des fichiers stockés dans le CephFS. Pour des raisons de performance, il est conforme aux normes POSIX et peut être interrogé à l'aide de programmes d'invite de commande Unix.

2.5 Le stockage des données dans le cluster Ceph

Lorsqu'une application veut stocker un objet dans l'infrastructure Ceph, elle va d'abord recevoir la cartographie de la distribution de l'infrastructure en premier lieu de la part du moniteur. Lorsqu'elle la reçoit, elle la compile. A l'issue de la compilation de la carte, elle saura exactement où elle doit stocker ses différents objets dans les OSD appropriés. Tout ceci est illustré dans la **Figure 2.6**.

Dans le cas où l'OSD en question se crache, le moniteur envoie la mise à jour de la carte de l'infrastructure au même client. Le client va compiler à nouveau la carte CRUSH pour savoir où il doit stocker ses objets.

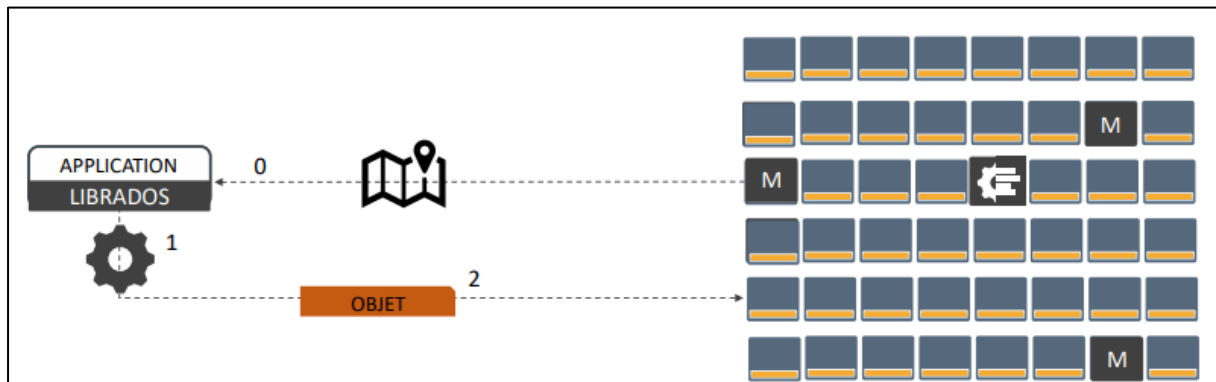


Figure 2.6 : Stockage des objets

2.6 La gestion des objets au niveau de l'infrastructure

Il peut y avoir différents types de données ça peut être un fichier, un film, une image, et autre. Prenons par exemple un film : Lorsque RADOS reçoit la requête de pouvoir stocker cette donnée, RADOS va fragmenter la donnée en plusieurs éléments qu'on appelle « Objet ». Ce sont ces objets que RADOS va manipuler. On n'est plus à l'étape de données mais à l'étape des objets. Ces objets seront associés à des pools. Il y'aura une fragmentation de ce pool en ce qu'on appelle « Groupe de placement » ou « PG ». C'est dans ce groupe de placement qu'on

aura notre objet. Ce PG va être copié plusieurs fois sur l'ensemble des OSD. C'est de cette façon que les différents objets seront stockés dans les OSDs. (Voir **Figure2.7**)

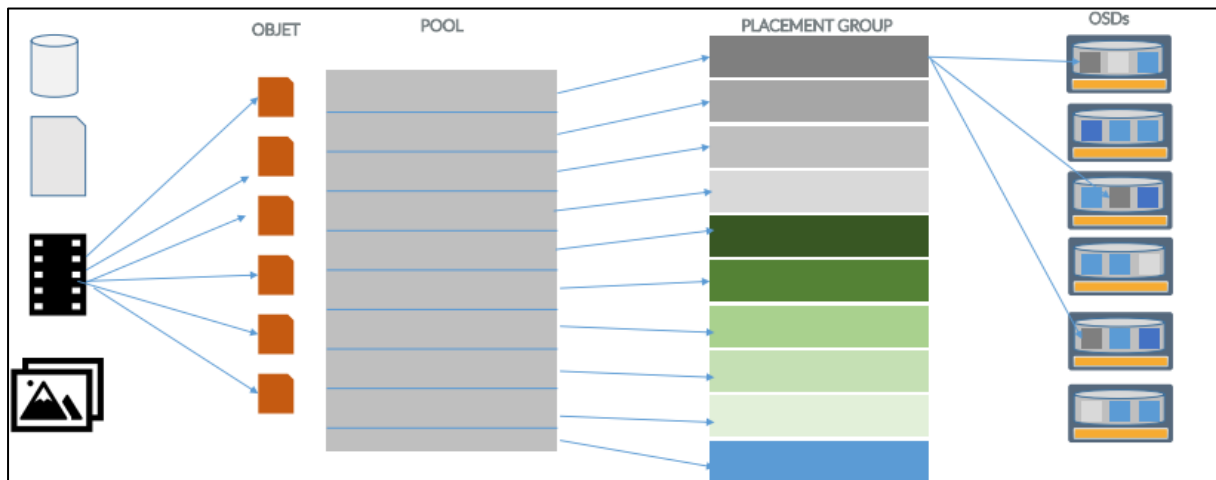


Figure 2.7 : Gestion des objets

2.7 Les modes de protection des données

Ceph propose deux modes de protection des données :

- ❖ **Réplication** : multiples copies de la source (voir **Figure2.8**)
 - Type de réplication le plus utilisé.
 - Réplication minimum par 3 pour réparer automatiquement les données détériorées.
 - Reconstruction rapide sans nécessité de calcul de parité.

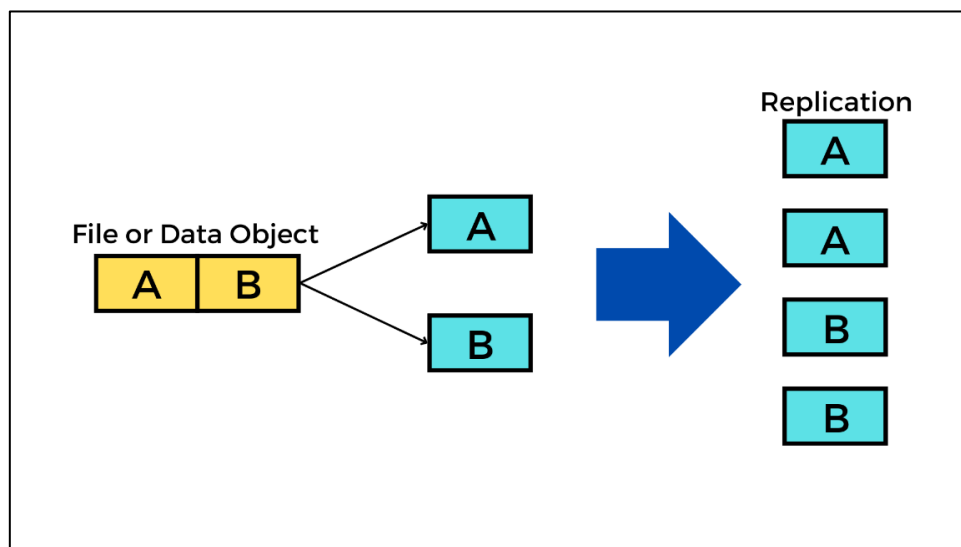


Figure 2.8 : Réplication des objets

❖ **Erasure-coding ou codage d'effacement** : codage et fragmentation de la source (voir **Figure 2.9**) :

- Les données sont divisées en K fragments, puis combinées et codées avec M morceaux de données redondantes réparties sur différents OSD du cluster.
- Protection équivalente à RAID6 : K=2, M=2.
- Différents algorithmes disponibles : Jerasure, ISA-l, LRC, shec.
- Reconstruction coûteuse en calcul.
- Les morceaux sont répartis sur des OSD de différents serveurs => Nécessite au minimum 4 serveurs pour débiter.

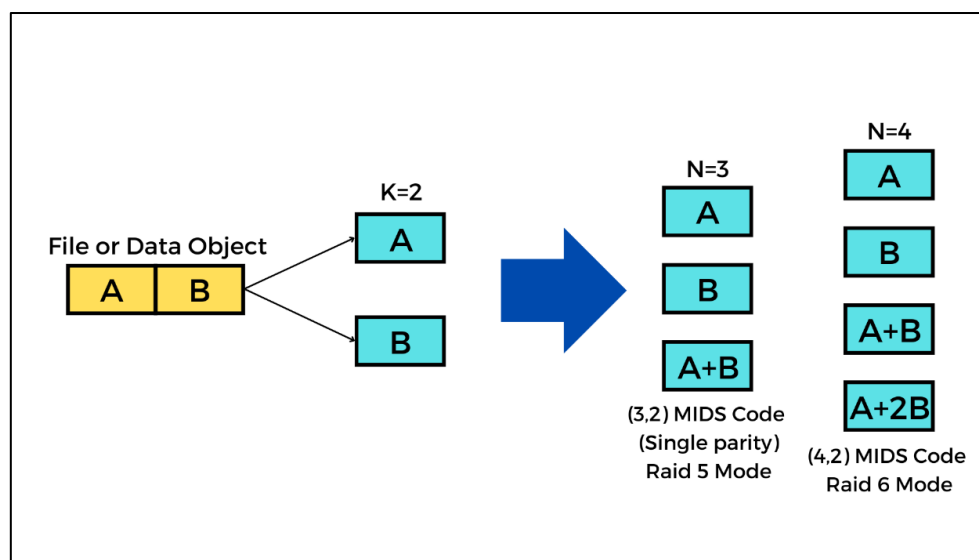


Figure 2.9 : Codage d'effacement

2.8 Les caractéristiques de Ceph

Ceph est devenu en quelques années la technologie qui compte dans le monde du stockage open source, en particulier du fait de ses caractéristiques, citons :

- ❖ **Fiabilité** :
 - Pas de SPOF (un point de défaillance unique).
 - Pérennité des données via le mode « Réplication » ou le mode « Codage d'effacement ».
 - Aucune interruption de service durant les mises à jour ou extension.
- ❖ **Cohérence et exactitude**.
- ❖ **Extensible (élasticité)** :
 - Facile augmentation ou diminution du cluster.

- Possibilité d'ajout ou de retrait d'équipement à chaud.
- ❖ Disponible en open source.
- ❖ Modularité importante.
- ❖ Sécurité des données grâce à une sauvegarde redondante des données.
- ❖ Augmentation de la disponibilité basée sur des logiciels grâce à un algorithme propre pour trouver les données.
- ❖ Affichage sous forme de mémoire liée.
- ❖ Faibles exigences matérielles (mise en place possible à partir d'1 Go de RAM sur un ordinateur avec un processeur Single Core et un espace disque de quelques Go en fonction de la tâche dans la grappe).

2.9 Conclusion

Ce chapitre, nous a permis d'avoir une vision globale et approfondie de ce qu'est Ceph et de comprendre son fonctionnement. Dans le chapitre suivant nous entamerons sa mise en place tout en expliquant le déroulement.

Chapitre 3 : Mise en place d'un cluster Ceph

3.1 Introduction

Ce chapitre constitue l'étape de mise en œuvre d'un cluster, c'est le côté pratique de ce qu'on a vu précédemment. Cela nous permettra de réaliser une partie de la solution finale. Durant cette phase, nous allons aborder la partie implémentation. Nous commencerons par présenter l'environnement matériel et logiciel et ensuite, les étapes de la réalisation.

3.2 Environnement de travail

Une bonne étude de l'environnement de travail aide à bien réaliser le projet. Pour cela, nous présentons les pré requis matériels et logiciels utilisés pour la mise en place du cluster ainsi que son architecture.

3.2.1 Architecture du cluster

L'architecture élaborée est l'architecture Client/Cluster, tel que le Client représente le consommateur de l'espace de stockage et le cluster c'est l'environnement Ceph. Elle est schématisée comme dans la **Figure 3.1**, ci-dessous.

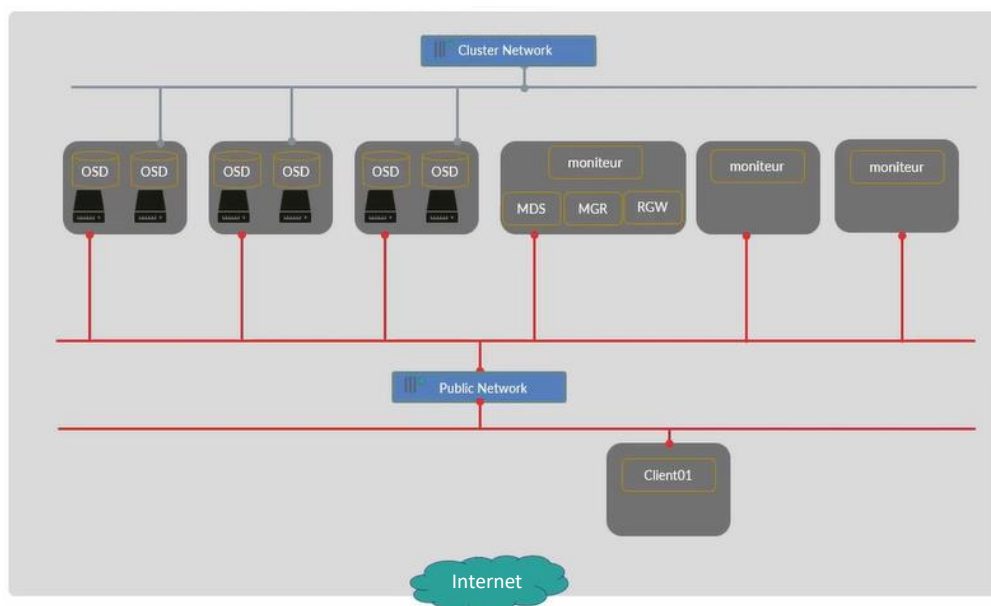


Figure 3.1 : Architecture du cluster Ceph

Ce cluster est composé de :

- ❖ Un réseau cluster utilisé par les OSDs pour la restauration, la répartition et le stockage des données.
- ❖ Un réseau public utilisé pour d'éventuel téléchargement et par le client pour le stockage.
- ❖ Trois serveurs OSDs pour le stockage des données.
- ❖ Trois serveurs moniteurs pour la gestion du cluster.
- ❖ Un client qui représente le consommateur de l'espace de stockage.

3.2.2 Matériels utilisés

Pour la réalisation du projet nous avons utilisé :

→ Deux ordinateurs portables dont les caractéristiques sont :

- ❖ Pour le premier :
 - Intel CORE i7-8550U CPU @ 1.80GHz 1.99 GHz.
 - 8 Go de mémoire vive.
 - Windows 10 - 64 bits.
- ❖ Pour le deuxième :
 - Intel CORE i7-10510U CPU @ 1.80GHz 2.30 GHz.
 - 16 Go de mémoire vive.
 - Windows 10 - 64 bits.

→ Un serveur AX41 Hetzner dont les caractéristiques sont :

- CPU de 6 Cores / 12 threads @ 3.6GHz.
- 64 Go de RAM.
- 4 To SSD.
- 4 Snapshots.
- 32 To de bande passante.
- CentOS 8 Stream.

3.2.3 Logiciels et outils utilisés

❖ KVM :

KVM (Kernel-based Virtual Machine) est une technologie de virtualisation Open Source intégrée à Linux. Avec KVM, on peut transformer Linux en un hyperviseur qui permet à une machine hôte d'exécuter plusieurs environnements virtuels isolés, appelés invités ou machines virtuelles. [\[10\]](#)



Figure 3.2 : Logo « KVM »

❖ PuTTY :

PuTTY est un émulateur permettant des connexions directes à des serveurs distants. Sa performance en matière de transfert est entre-autre assurée par la combinaison de clients SSH et Telnet. Il ne nécessite aucune installation. C'est un logiciel totalement libre et il est disponible en plusieurs langues. [\[11\]](#)

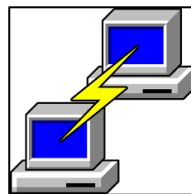


Figure 3.3 : Logo « PuTTY »

❖ Cephadm :

Cephadm est un outil de déploiement qui est là pour faciliter le déploiement des clusters Ceph. Il utilise Podman pour exécuter tous les démons Ceph, et la gestion du déploiement se fait via une connexion SSH. L'exécution de Ceph avec des conteneurs présente de nombreux avantages, principalement lors d'une mise à niveau, d'échecs, etc. Pour commencer à utiliser cephadm tout ce dont on a besoin est de quelques machines (physiques/virtuelles), de quelques packages et d'une connexion SSH entre le déploieur et les machines du cluster Ceph.

3.3 La réalisation

Dans cette partie, nous déployons le cluster Ceph illustré dans la **Figure 3.1**, après avoir suivi la formation [12]. Nous nous sommes également appuyés de la documentation du site officiel de Ceph [13].

3.3.1 Virtualisation du server physique : Installation de KVM

- ❖ Vérifier que le CPU supporte la virtualisation :

```
# egrep -c '(vmx|svm)' /proc/cpuinfo
```

- ❖ Installer KVM et les paquets de prise en charge :

```
# yum install qemu-kvm libguestfs-tools virt-install -y
```

- ❖ Activer le démarrage automatique et lancer le démon de gestion de la virtualisation :

```
# systemctl enable libvirtd && systemctl start libvirtd
```

- ❖ Vérifier que le module KVM est bien chargé :

```
# lsmod | grep -i kvm
```

3.3.2 Déploiement de l'environnement Ceph cluster

a) Création du réseau public et cluster

- ❖ **Réseau public :**

- Initialisation du réseau public « cephpublicnet ». L'interface sera configurée en mode NAT et devra passer par l'interface physique pour avoir accès à internet pour le téléchargement des paquets ou autres.

```
# vi /root/cephpublicnet.xml
```

```
<network>
  <name>cephpublicnet</name>
  <bridge name="cephpublicnet"/>
    <forward mode="nat" dev='enp41s0'/>
    <ip address="192.168.30.1" netmask="255.255.255.0">
      <dhcp>
        <range start="192.168.30.2" end="192.168.30.254"/>
      </dhcp>
    </ip>
</network>
```

Figure 3.4 : Initialisation du réseau public

- Procéder à la création du réseau « cephpublicnet » :

```
# virsh net-define /root/cephpublicnet.xml
```

```
[root@rootserver01 ~]# virsh net-define /root/cephpublicnet.xml
Network cephpublicnet defined from /root/cephpublicnet.xml
```

Figure 3.5 : Création du réseau public

- Activer le démarrage automatique du réseau « cephpublicnet » :

```
# virsh net-autostart cephpublicnet
```

```
[root@rootserver01 ~]# virsh net-autostart cephpublicnet
Network cephpublicnet marked as autostarted
```

Figure 3.6 : Activation du démarrage automatique du réseau public

- Démarrer le réseau « cephpublicnet » :

```
# virsh net-start cephpublicnet
```

```
[root@rootserver01 ~]# virsh net-start cephpublicnet
Network cephpublicnet started
```

Figure 3.7 : Démarrage du réseau public

❖ **Réseau cluster :**

- Initialisation du réseau cluster « cephclusternet ». Le switch aura l'adresse 192.168.15.1 et les autres VM seront dynamiquement assignés à la plage d'adresse [192.168.15.2 – 192.168.15.254]

```
# vi /root/cephclusternet.xml
```

```
<network>
  <name>cephclusternet</name>
  <bridge name="cephclusternet" stp='on' delay='0' />
  <ip address="192.168.15.1" netmask="255.255.255.0">
    <dhcp>
      <range start="192.168.15.2" end="192.168.15.254" />
    </dhcp>
  </ip>
</network>
```

Figure 3.8 : Initialisation du réseau cluster

- Procéder à la création du réseau « cephclusternet » :

```
# virsh net-define /root/cephclusternet.xml
```

```
[root@rootserver01 ~]# virsh net-define /root/cephclusternet.xml
Network cephclusternet defined from /root/cephclusternet.xml
```

Figure 3.9 : Création du réseau cluster

- Activer le démarrage automatique du réseau « cephclusternet » :

```
# virsh net-autostart cephclusternet
```

```
[root@rootserver01 ~]# virsh net-autostart cephclusternet
Network cephclusternet marked as autostarted
```

Figure 3.10 : Activation du démarrage automatique du réseau cluster

- Démarrer le réseau « cephclusternet » :

```
# virsh net-start cephclusternet
```

```
[root@rootserver01 ~]# virsh net-start cephclusernet
Network cephclusernet started
```

Figure 3.11 : Démarrage du réseau cluster

❖ **Lister les réseaux virtuels :**

- Lister les réseaux virtuels avec la commande : « # **virsh net-list --all** » :

```
[root@rootserver01 ~]# virsh net-list --all
```

Name	State	Autostart	Persistent
cephclusernet	active	yes	yes
cephpublicnet	active	yes	yes
management01	active	yes	yes
provider01	active	yes	yes

Figure 3.12 : Liste des réseaux virtuels libvirt actifs

b) Déploiement des nœuds templates

❖ Télécharger CentOS 8 Stream :

```
# curl -O http://mirror.marwan.ma/centos/8-stream/isos/x86_64/CentOS-Stream-8-x86_64-20220603-boot.iso
```

❖ Vérifier que l'installation s'est bien effectuée grâce à la commande « # **ls -ltr** » :

```
[root@rootserver01 ~]# ls -ltr
total 3250220
-rw-r--r-- 1 root root      259 May  9 12:14 management01.xml
-rw-r--r-- 1 root root      185 May  9 12:14 provider01.xml
-rw-r--r-- 1 qemu qemu 1020264448 May  9 12:18 CentOS-7-x86_64-Minimal-2009.iso
-rw-r--r-- 1 root root      231 May 16 22:41 cephclusernet01.xml
-rw-r--r-- 1 root root      227 May 16 22:53 cephclusernet.xml
-rw-r--r-- 1 root root      267 May 16 22:55 cephpublicnet.xml
-rw-r--r-- 1 root root  758120448 May 22 09:01 CentOS-8.4.2105-x86_64-boot.iso
-rw-r--r-- 1 root root  778043392 Jun  4 22:28 CentOS-Stream-8-x86_64-20220603-boot.iso
-rw-r--r-- 1 root root  771751936 Jun  7 22:07 CentOS-Stream-8-x86_64-20220603-boot.iso
-rw-r--r-- 1 root root      252 Jun 10 22:11 ceph_cluserNet01.xml
-rw-r--r-- 1 root root      243 Jun 10 22:31 cluster_network.xml
```

Figure 3.13 : Affichage du fichier téléchargé

❖ **Création du dépôt :**

- Installer Apache :

```
# yum install httpd -y
```

- Créer un repository local comme point de montage :

```
# mkdir /mnt/CentOS8Stream-install/
```

- Monter le fichier iso sur le repository :

```
# mount -o loop,ro -t iso9660 /root/CentOS-Stream-8-x86_64-20220603-boot.iso  
/mnt/CentOS8-install/
```

- Recopier l'ensemble des données du fichier iso dans le répertoire html d'apache :

```
# cp -r /mnt/CentOS8Stream-install/ /var/www/html/
```

- Démarrer le service Apache :

```
# systemctl start httpd.service
```

- Vérifier que le service Apache fonctionne bien :

```
# systemctl status httpd
```

```
[root@rootserver01 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; vendor preset: disabled)
   Active: active (running) since Sun 2022-05-09 12:21:15 CEST; 5s ago
     Docs: man:httpd(8)
           man:apachectl(8)
  Main PID: 8723 (httpd)
    Status: "Processing requests..."
     Tasks: 6
    CGroup: /system.slice/httpd.service
            └─8723 /usr/sbin/httpd -DFOREGROUND
              └─8724 /usr/sbin/httpd -DFOREGROUND
                └─8725 /usr/sbin/httpd -DFOREGROUND
                  └─8726 /usr/sbin/httpd -DFOREGROUND
                    └─8727 /usr/sbin/httpd -DFOREGROUND
                      └─8728 /usr/sbin/httpd -DFOREGROUND
```

Figure 3.14 : Affichage de l'état du serveur Apache

- Vérifier que tout fonctionne bien avec la commande suivante :

```
# curl -I http://192.168.30.1/CentOS8Stream-install/
```

```
[root@rootserver01 html]# curl -I http://192.168.30.1/CentOS8Strem-install/
HTTP/1.1 200 OK
Date: Tue, 14 Jun 2022 12:04:52 GMT
Server: Apache/2.4.6 (CentOS)
Content-Type: text/html; charset=ISO-8859-1
```

Figure 3.15 : Récupération des informations du site http

c) Clonage des nœuds pour obtenir 3 OSD, 3 MON et un client

On va déployer deux nœuds templates qui seront clonés par la suite : un nœud OSD et un nœud MON.

❖ Déployer le template moniteur :

- Installation et configuration du nœud avec cette commande :

```
# virt-install \
--name centos8tempmon \
--memory 4096 \
--vcpus 2 \
--disk size=50 \
--location http://192.168.221.130/CentOS8Stream-install /\
--os-variant centos-stream8 \
--nographics \
--extra-args="console=ttyS0,115200n8" \
--network network:cephpublicnet
```

❖ **Déployer le template OSD :**

- Installation et configuration du nœud avec cette commande :

```
# virt-install \
--name tempOSd \
--memory 4096 \
--vcpus 2 \
--disk size=50 \
--location http://192.168.221.130/CentOS8Stream-install/ \
--os-variant rhel8 \
--nographics \
--extra-args="console=ttyS0,115200n8" \
--network network:cephpublicnet \
--network network:cephclusternet
```

❖ **Afficher les adresses assignées aux nœuds :**

- Se connecter au nœud en question avec la commande « #ssh »
- Exécuter la commande « # ip addr » pour afficher les adresses.

❖ **Vérifier que nos deux serveurs ont été ajoutés :** avec la commande « # virsh list --all »
on remarque que nos deux serveurs ont été ajoutés.

```
[root@rootserver01 ~]# virsh list --all
 Id      Name                                State
-----
 99      deploynode                         running
100      blockstorage01                     running
101      compute01                          running
104      openstackAllinOne                  running
123      centos8tempmon                      running
124      tempOSd                            running
-       centos8temp                        shut off
-       centOSSteam                         shut off
-       controller                         shut off
-       mon1                               shut off
```

Figure 3.16 : Liste de toutes les machines actives et inactives

❖ Cloner les serveurs templates :

- Arrêter les serveurs templates pour le moniteur et pour l'osd :

```
# virsh destroy centos8tempmon  
# virsh destroy tempOSd
```

```
[root@rootserver01 ~]# virsh destroy centos8tempmon  
Domain centos8tempmon destroyed
```

Figure 3.17 : Arrêt du serveur centos8tempmon

```
[root@rootserver01 ~]# virsh destroy tempOSd  
Domain tempOSd destroyed
```

Figure 3.18 : Arrêt du serveur tempOSd

- Cloner les machines virtuelles pour créer le mon1, mon2, mon3, osd1, osd2, osd3 et le client :

```
# cd /var/lib/libvirt/images  
# virt-clone --original centos8tempmon --name mon1 --auto-clone  
# virt-clone --original centos8tempmon --name mon2 --auto-clone  
# virt-clone --original centos8tempmon --name mon3 --auto-clone  
# virt-clone --original centos8temp --name osd1 --auto-clone  
# virt-clone --original centos8temp --name osd2 --auto-clone  
# virt-clone --original centos8temp --name osd3 --auto-clone  
# virt-clone --original centos8tempmon --name client1 --auto-clone
```

```
[root@rootserver01 images]# virt-clone --original centos8tempmon --name mon1 --auto-clone
Allocating 'mon1.qcow2' | 50 GB 00:01:11

Clone 'mon1' created successfully.
[root@rootserver01 images]# virt-clone --original centos8tempmon --name mon2 --auto-clone
Allocating 'mon2.qcow2' | 50 GB 00:00:04

Clone 'mon2' created successfully.
[root@rootserver01 images]# virt-clone --original centos8tempmon --name mon3 --auto-clone
Allocating 'mon3.qcow2' | 50 GB 00:00:16

Clone 'mon3' created successfully.
[root@rootserver01 images]# virt-clone --original tempOSd --name osd1 --auto-clone
Allocating 'osd1.qcow2' | 50 GB 00:01:39

Clone 'osd1' created successfully.
[root@rootserver01 images]# virt-clone --original tempOSd --name osd2 --auto-clone
Allocating 'osd2.qcow2' | 50 GB 00:00:04

Clone 'osd2' created successfully.
[root@rootserver01 images]# virt-clone --original tempOSd --name osd3 --auto-clone
Allocating 'osd3.qcow2' | 50 GB 00:00:16

Clone 'osd3' created successfully.
[root@rootserver01 images]# virt-clone --original centos8tempmon --name client1 --auto-clone
Allocating 'client1.qcow2' | 50 GB 00:01:13

Clone 'client1' created successfully.
[root@rootserver01 images]#
```

Figure 3.19 : Clonage des machines virtuelles

d) Attachement des 6 disques virtuels aux nœuds OSDs

❖ Créer les disques pour l'osd1 sur le KVM :

```
# qemu-img create -f raw /var/lib/libvirt/images/ disk1osd1.img 30G
# qemu-img create -f raw /var/lib/libvirt/images/ disk2osd1.img 30G
```

❖ Attacher les disques à l'osd1 :

```
# virsh attach-disk osd1 --source /var/lib/libvirt/images/disk1osd1.img --target sda --
persistent
# virsh attach-disk osd1 --source /var/lib/libvirt/images/disk2osd1.img --target sdb --
persistent
```

- ❖ Créer les disques pour l'osd2 sur le KVM :

```
# qemu-img create -f raw /var/lib/libvirt/images/disk1osd2.img 30G
# qemu-img create -f raw /var/lib/libvirt/images/disk2osd2.img 30G
```

- ❖ Attacher les disques à l'osd2 :

```
# virsh attach-disk osd2 --source /var/lib/libvirt/images/disk1osd2.img --target sda --
persistent
# virsh attach-disk osd2 --source /var/lib/libvirt/images/disk2osd2.img --target sdb --
persistent
```

- ❖ Créer les disques pour l'osd3 sur le KVM :

```
# qemu-img create -f raw /var/lib/libvirt/images/disk1osd3.img 30G
# qemu-img create -f raw /var/lib/libvirt/images/disk2osd3.img 30G
```

- ❖ Attacher les disques à l'osd3 :

```
# virsh attach-disk osd3 --source /var/lib/libvirt/images/ disk1osd3.img --target sda --
persistent
# virsh attach-disk osd3 --source /var/lib/libvirt/images/ disk2osd3.img --target sdb --
persistent
```

❖ Démarrer les serveurs :

```
# virsh start mon1
# virsh start mon2
# virsh start mon3
# virsh start osd1
# virsh start osd2
# virsh start osd3
# virsh start client1
```

```
[root@rootserver01 images]# virsh start mon1
Domain mon1 started

[root@rootserver01 images]# virsh start mon2
Domain mon2 started

[root@rootserver01 images]# virsh start mon3
Domain mon3 started

[root@rootserver01 images]# virsh start osd1
Domain osd1 started

[root@rootserver01 images]# virsh start osd2
Domain osd2 started

[root@rootserver01 images]# virsh start osd3
Domain osd3 started

[root@rootserver01 images]# virsh start client1
Domain client1 started
```

Figure 3.20 : Démarrage des machines virtuelles

❖ Afficher les adresses des serveurs :

```
# virsh domifaddr mon1
# virsh domifaddr mon2
# virsh domifaddr mon3
# virsh domifaddr osd1
# virsh domifaddr osd2
# virsh domifaddr osd3
# virsh domifaddr client1
```

```
[root@rootserver01 images]# virsh domifaddr mon1
Name          MAC address          Protocol    Address
-----
vnet3         52:54:00:86:50:17      ipv4       192.168.30.230/24

[root@rootserver01 images]# virsh domifaddr mon2
Name          MAC address          Protocol    Address
-----
vnet5         52:54:00:bb:fe:dc      ipv4       192.168.30.10/24

[root@rootserver01 images]# virsh domifaddr mon3
Name          MAC address          Protocol    Address
-----
vnet7         52:54:00:16:ae:75      ipv4       192.168.30.208/24

[root@rootserver01 images]# virsh domifaddr osd1
Name          MAC address          Protocol    Address
-----
vnet8         52:54:00:d6:2d:7c      ipv4       192.168.30.150/24
vnet9         52:54:00:bc:a4:ac      ipv4       192.168.15.22/24

[root@rootserver01 images]# virsh domifaddr osd2
Name          MAC address          Protocol    Address
-----
vnet10        52:54:00:a3:7f:41      ipv4       192.168.30.52/24
vnet11        52:54:00:eb:2e:93      ipv4       192.168.15.181/24

[root@rootserver01 images]# virsh domifaddr osd3
Name          MAC address          Protocol    Address
-----
vnet12        52:54:00:c1:db:97      ipv4       192.168.30.209/24
vnet13        52:54:00:70:7c:03      ipv4       192.168.15.111/24

[root@rootserver01 images]# virsh domifaddr client1
Name          MAC address          Protocol    Address
-----
vnet14        52:54:00:15:ff:26      ipv4       192.168.30.99/24
```

Figure 3.21 : Adresses IP des machines virtuelles

3.3.3 Installation des prérequis de Ceph

a) Dans le serveur

- ❖ Sauvegarder les adresses des différentes machines dans le répertoire « **/etc/hosts** » :

```
# echo -e "192.168.30.230 mon1\n192.168.30.10 \ mon2\n192.168.30.208
mon3\n192.168.30.150 \ osd1\n192.168.30.52 osd2\n192.168.30.209
osd3\n192.168.30.123 \ 192.168.30.99" >> /etc/hosts
```

b) Dans le mon1

- ❖ Accéder au mon1 avec la commande « **# ssh mon1** » :

```
[root@rootserver01 images]# ssh mon1
Warning: Permanently added the ECDSA host key for IP address '192.168.30.230' to the list of
known hosts.
root@mon1's password:
Activate the web console with: systemctl enable --now cockpit.socket
```

Figure 3.22 : Connection SSH au mon1

- ❖ Sauvegarder les adresses des différentes machines dans le répertoire « **/etc/hosts** » :

```
# echo -e "192.168.30.230 mon1\n192.168.30.10 \ mon2\n 192.168.30. 208
mon3\n192.168.30.150 \ osd1\n192.168.30.52 osd2\n 192.168.30. 209
osd3\n192.168.30.123 \ 192.168.30.99" >> /etc/hosts
```

- ❖ Modifier le hostname avec la commande « **# hostnamectl set-hostname mon1** »
- ❖ Mettre à jour les paquets avec la commande « **# yum update -y** ».
- ❖ Arrêter le firewalld :

```
# systemctl stop firewalld && systemctl disable firewalld
```

```
[root@localhost ~]# systemctl stop firewalld && systemctl disable firewalld
Removed /etc/systemd/system/multi-user.target.wants/firewalld.service.
Removed /etc/systemd/system/dbus-org.fedoraproject.FirewallD1.service.
```

Figure 3.23 : Arrêter et désactiver Firewalld

❖ Installer le Docker :

- Installer les paquets nécessaires :

```
# yum install -y yum-utils
```

```
[root@localhost ~]# yum install -y yum-utils
Last metadata expiration check: 1 day, 14:49:38 ago on Tue 14 Jun 2022 12:35:14 PM EDT.
Dependencies resolved.
=====
Package                Architecture      Version           Repository        Size
=====
Installing:
yum-utils              noarch           4.0.21-1.el8     baseos            72 k
Transaction Summary
=====
Install 1 Package
```

Figure 3.24 : Installation du paquet yum-utils

- Ajouter le lien du docker au niveau du repository :

```
# yum-config-manager \
--add-repo \
https://download.docker.com/linux/centos/docker-ce.repo
```

```
[root@localhost ~]# yum-config-manager \
> --add-repo \
> https://download.docker.com/linux/centos/docker-ce.repo
Adding repo from: https://download.docker.com/linux/centos/docker-ce.repo
```

Figure 3.25 : Configuration du repository

- Installer le docker : On met l'extension « allowrasing » car il y a par défaut le Podman qui est aussi une plateforme de conteneurisation. Il va supprimer Podman et installer docker.

```
# yum install docker-ce docker-ce-cli containerd.io -y --allowrasing
```

```
[root@localhost ~]# sudo yum install docker-ce docker-ce-cli containerd.io -y --allowdowngrade
Docker CE Stable - x86_64
Dependencies resolved.
=====
Package Arch Version Repository Size
=====
Installing:
containerd.io x86_64 1.4.6-3.1.el8 docker-ce-stable 34 M
replacing runc.x86_64 1.0.0-70.rc92.module_el8.5.0+733+9bb5dffa
docker-ce x86_64 3:20.10.7-3.el8 docker-ce-stable 27 M
docker-ce-cli x86_64 1:20.10.7-3.el8 docker-ce-stable 33 M
Installing dependencies:
docker-ce-rootless-extras
docker-scan-plugin x86_64 0.8.0-3.el8 docker-ce-stable 4.2 M
libcgroup x86_64 0.41-19.el8 baseos 70 k
Removing dependent packages:
buildah x86_64 1.19.8-1.module_el8.5.0+733+9bb5dffa @appstream 25 M
cockpit-podman noarch 29-3.module_el8.5.0+733+9bb5dffa @appstream 3.8 M
podman x86_64 3.1.0-0.13.module_el8.5.0+733+9bb5dffa @appstream 46 M
podman-catatonit x86_64 3.1.0-0.13.module_el8.5.0+733+9bb5dffa @appstream 760 k
Transaction Summary
=====
Install 6 Packages
Remove 4 Packages
```

Figure 3.26 : Installation de Docker

- Démarrer notre docker et par la même occasion l'activer au démarrage du serveur :

```
# systemctl start docker && systemctl enable docker
```

```
[root@localhost ~]# sudo systemctl start docker && sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
```

Figure 3.27 : Démarrage et activation de Docker

- ❖ Installer python3 avec la commande « **# yum install -y python3** » : Par défaut python3 est déjà installé.

```
[root@localhost ~]# yum install -y python3
Last metadata expiration check: 0:00:46 ago on Tue 14 Jun 2022 03:25:12 AM EDT.
Package python36-3.6.8-37.module_el8.5.0+771+e5d9a225.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
```

Figure 3.28 : Installation de Python3

- ❖ Installer NTP avec la commande « **# yum install chrony** » : Par défaut NTP est déjà installé. Donc on va le configurer seulement :
 - Configurer NTP : Accéder à **/etc/chrony.conf** avec la commande « **# vi /etc/chrony.conf** » : On ouvre le réseau public afin que tous les serveurs OSDs puissent s'y connecter « **allow 192.168.30.0/24** »

```
# Allow NTP client access from local network.
allow 192.168.30.0/24
```

Figure 3.29 : Configuration de NTP

- Redémarrer le service chrony :

```
# systemctl restart chronyd.service
```

- Vérifier que tout marche avec la commande « **# chronyc sources** » :

```
[root@localhost ~]# chronyc sources
MS Name/IP address          Stratum Poll Reach LastRx Last sample
=====
^- time01.nevondo.com        2    6    7    2   -275us[ -693us] +/-  41ms
^- bachschmoltz.de           2    6    7    2   +2383us[+1965us] +/-  60ms
^+ time.cloudflare.com       3    6    7    2   +205us[ -213us] +/-  18ms
^* mail.zql.de               2    6    7    2   -298us[ -716us] +/-  14ms
[root@localhost ~]#
```

Figure 3.30 : Affichage des sources NTP

- ❖ Répéter les mêmes étapes pour mon2 et mon3.

c) Dans l'osd1

- ❖ Accéder à l'osd1 avec la commande « **# ssh osd1** » :

```
[root@mon1 ~]# ssh osd1
The authenticity of host 'osd1 (192.168.30.150)' can't be established.
ECDSA key fingerprint is SHA256:5XbvJ284AeP+UEIpNGFoYibBALYcscLUP5VS1o5Vb80.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'osd1,192.168.30.150' (ECDSA) to the list of known hosts.
root@osd1's password:
Activate the web console with: systemctl enable --now cockpit.socket
```

Figure 3.31 : Connexion SSH à l'osd1

- ❖ Sauvegarder les adresses des différentes machines dans le répertoire « **/etc/hosts** » :

```
# echo -e "192.168.30.230 mon1\n192.168.30.10 \ mon2\n192.168.30. 208
mon3\n192.168.30.150 \ osd1\n192.168.30.52 osd2\n192.168.30. 209 osd3\n192.168.30.123 \
192.168.30.99" >> /etc/hosts
```

- ❖ Modifier le hostname avec la commande « **# hostnamectl set-hostname osd1** »
- ❖ Mettre à jour les paquets avec la commande « **# yum update -y** ».
- ❖ Arrêter le firewalld :

```
# systemctl stop firewalld && systemctl disable firewalld
```

```
[root@localhost ~]# systemctl stop firewalld && systemctl disable firewalld
Removed /etc/systemd/system/multi-user.target.wants/firewalld.service.
Removed /etc/systemd/system/dbus-org.fedoraproject.FirewallD1.service.
```

Figure 3.32 : Arrêter et désactiver Firewalld

❖ Installer le Docker :

- Installer les paquets nécessaires :

```
# yum install -y yum-utils
```

```
[root@localhost ~]# yum install -y yum-utils
Last metadata expiration check: 1 day, 14:49:38 ago on Tue 14 Jun 2022 12:35:14 PM EDT.
Dependencies resolved.
=====
Package                Architecture      Version           Repository        Size
=====
Installing:
yum-utils              noarch           4.0.21-1.el8     baseos            72 k
Transaction Summary
=====
Install 1 Package
```

Figure 3.33 : Installation du paquet yum-utils

- Ajouter le lien du docker au niveau du repository :

```
# yum-config-manager \
--add-repo \
https://download.docker.com/linux/centos/docker-ce.repo
```

```
[root@localhost ~]# yum-config-manager \
> --add-repo \
> https://download.docker.com/linux/centos/docker-ce.repo
Adding repo from: https://download.docker.com/linux/centos/docker-ce.repo
```

Figure 3.34 : Configuration du repository

- Installer le docker :

```
# yum install docker-ce docker-ce-cli containerd.io -y --allowdowngrade
```

```
[root@localhost ~]# sudo yum install docker-ce docker-ce-cli containerd.io -y --allowdowngrade
Docker CE Stable - x86_64
Dependencies resolved.
=====
Package Arch Version Repository Size
=====
Installing:
containerd.io x86_64 1.4.6-3.1.el8 docker-ce-stable 34 M
replacing runc.x86_64 1.0.0-70.rc92.module_el8.5.0+733+9bb5dffa
docker-ce x86_64 3:20.10.7-3.el8 docker-ce-stable 27 M
docker-ce-cli x86_64 1:20.10.7-3.el8 docker-ce-stable 33 M
Installing dependencies:
docker-ce-rootless-extras
docker-scan-plugin x86_64 0.8.0-3.el8 docker-ce-stable 4.2 M
libcgroup x86_64 0.41-19.el8 baseos 70 k
Removing dependent packages:
buildah x86_64 1.19.8-1.module_el8.5.0+733+9bb5dffa @appstream 25 M
cockpit-podman noarch 29-3.module_el8.5.0+733+9bb5dffa @appstream 3.8 M
podman x86_64 3.1.0-0.13.module_el8.5.0+733+9bb5dffa @appstream 46 M
podman-catatonit x86_64 3.1.0-0.13.module_el8.5.0+733+9bb5dffa @appstream 760 k
Transaction Summary
=====
Install 6 Packages
Remove 4 Packages
```

Figure 3.35 : Installation de Docker

- Démarrer notre docker et par la même occasion l'activer au démarrage du serveur :

```
# systemctl start docker && systemctl enable docker
```

```
[root@localhost ~]# sudo systemctl start docker && sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
```

Figure 3.36 : Démarrage et activation de Docker

- ❖ Installer python3 avec la commande « **# yum install -y python3** » : Par défaut python3 est déjà installé.
- ❖ Installer NTP avec la commande « **# yum install chrony** » : Par défaut NTP est déjà installé. Donc on va le configurer seulement :
 - Configurer le NTP : L'osd doit pouvoir pointer sur les trois moniteurs. Pour cela, on accède à **/etc/chrony.conf** avec la commande « **# vi /etc/chrony.conf** » : On ajoute ces trois lignes :
 - **server cephmon01 iburst**
 - **server cephmon02 iburst**
 - **server cephmon03 iburst**

```
#pool 2.centos.pool.ntp.org iburst
server mon1 iburst
server mon2 iburst
server mon3 iburst
# Record the rate at which the system clock gains/losses time.
driftfile /var/lib/chrony/drift
```

Figure 3.37 : Configuration de NTP pour l'osd1

- Redémarrer le service chrony :

```
# systemctl restart chronyd.service
```

- Vérifier que tout marche avec la commande « **# chronyc sources** » :

```
MS Name/IP address         Stratum Poll Reach LastRx Last sample
=====
^* mon1                    3      6   17    4    +16us[ +15us] +/- 15ms
^? mon2                    0      7    0    -     +0ns[ +0ns] +/-  0ns
^? mon3                    0      7    0    -     +0ns[ +0ns] +/-  0ns
```

Figure 3.38 : Affichage des sources NTP

- ❖ Répéter les mêmes étapes pour osd2 et osd3.

3.3.4 Installation de Ceph avec l'outil Cephadm

Nous allons effectuer cette installation depuis le serveur mon1 qui sera notre serveur d'administration et de déploiement.

- ❖ Accéder au mon1 avec la commande « **# ssh mon1** »
- ❖ Récupérer la version la plus récente du script autonome :

```
# curl -silent --remote-name --location https://github.com/ceph/ceph/raw/pacific/src/cephadm/cephadm
```

- ❖ Rendre le script autonome exécutable :

```
# chmod +x cephadm
```

- ❖ Installer les paquets Cephadm :

```
# ./cephadm add-repo --release pacific  
# ./cephadm install
```

- ❖ Installer les différentes commandes Ceph sur notre environnement :

```
# cephadm install ceph-common
```

- ❖ Créer le répertoire **/etc/ceph** qui va recevoir tous les fichiers de configuration :

```
# mkdir -p /etc/ceph
```

- ❖ Lancer l'installation de notre moniteur :

```
# cephadm bootstrap --mon-ip 192.168.30.230
```

- ❖ De la dernière exécution on obtient un URL, un username et un password : On copie le lien sur le navigateur et on fait rentrer nos coordonnées. Nous sommes priés de changer le mot de passe de Ceph. Une fois que c'est fait, un dashboard s'affiche permettant de voir tout notre cluster et de le gérer.

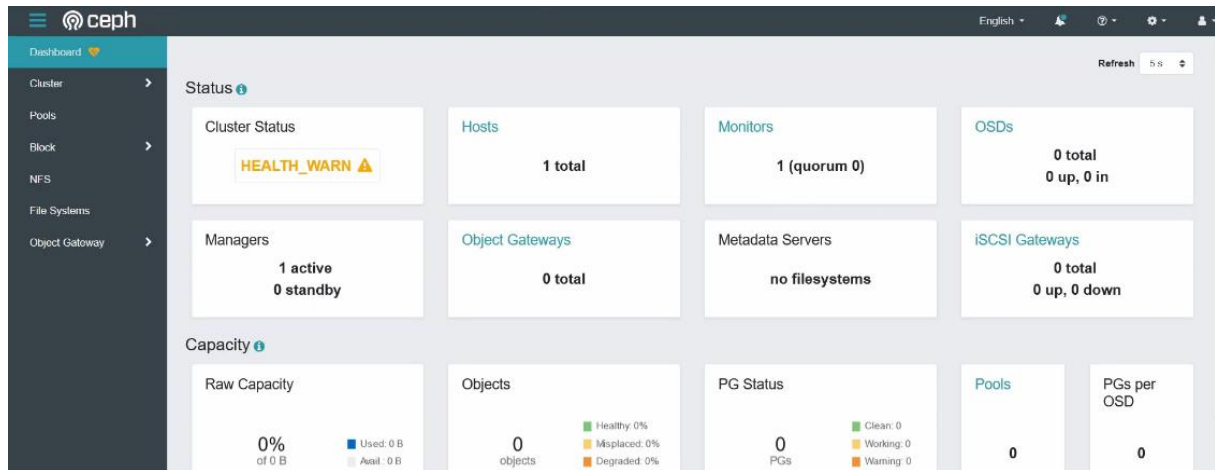


Figure 3.39 : Interface graphique de Ceph

- ❖ Ajouter un tunnel pour ouvrir une nouvelle session qui va établir une connexion entre le serveur physique et les nœuds virtuelles.
- ❖ Définir le réseau cluster et le réseau public :

```
# ceph config set global cluster_network 192.168.15.0/24
# ceph config set global public_network 192.168.30.0/24
```

- ❖ Redémarrer toute l'infrastructure pour que les informations soient prises en compte :

```
# systemctl restart ceph\*.service ceph\*.target
```

- ❖ Afficher l'état du cluster avec la commande « **#ceph -w** » :

```
[root@mon1 ~]# ceph -w
cluster:
  id:      82abfb68-e540-11eb-aba7-525400865017
  health: HEALTH_WARN
           OSD count 0 < osd_pool_default_size 3

services:
  mon: 1 daemons, quorum mon1 (age 23s)
  mgr: mon1.cpopbw(active, since 18s)
  osd: 0 osds: 0 up, 0 in

data:
  pools:   0 pools, 0 pgs
  objects: 0 objects, 0 B
  usage:   0 B used, 0 B / 0 B avail
  pgs:
```

Figure 3.40 : Etat du cluster Ceph

- ❖ Ajouter un label au moniteur :

```
# ceph orch host label add mon1 mon
# ceph orch apply mon mon1
```

- ❖ Lister l'ensemble des serveurs qu'il y a sur la plateforme avec la commande « **# ceph orch host ls** » :

```
[root@mon1 ~]# ceph orch host ls
HOST      ADDR          LABELS          STATUS
mon1      192.168.30.230 _admin mon
```

Figure 3.41 : Liste des hôtes sur l'invite de commande

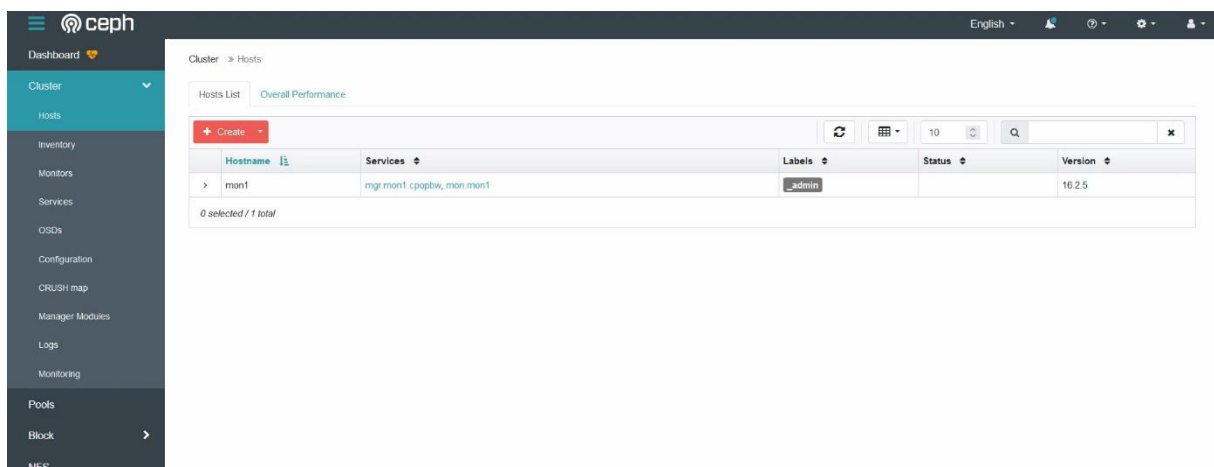


Figure 3.42 : Liste des hôtes sur l'interface graphique

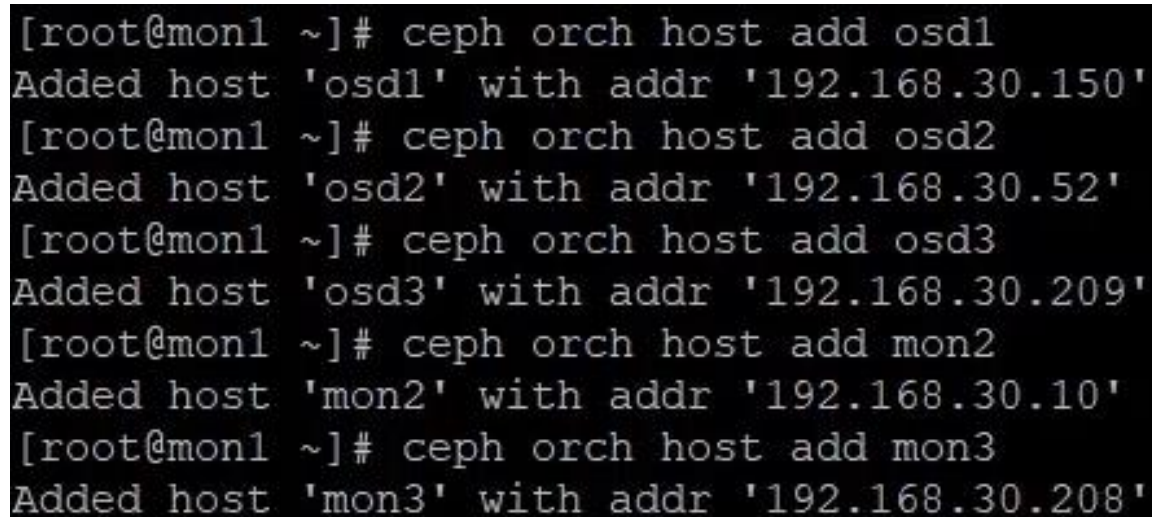
❖ Ajouter des nœuds :

- A partir de la clé publique, nous allons nous connecter aux différents autres serveurs afin d'établir un lien de confiance entre ces serveurs et le serveur mon1 :

```
# ssh-copy-id -f -i /etc/ceph/ceph.pub osd1
# ssh-copy-id -f -i /etc/ceph/ceph.pub osd2
# ssh-copy-id -f -i /etc/ceph/ceph.pub osd3
# ssh-copy-id -f -i /etc/ceph/ceph.pub mon2
# ssh-copy-id -f -i /etc/ceph/ceph.pub mon3
```

- Le lien de confiance a été établi entre le serveur mon1 et les autres serveurs de l'infrastructure donc on peut passer à leur ajout dans le cluster :

```
# ceph orch host add osd1
# ceph orch host add osd2
# ceph orch host add osd3
# ceph orch host add mon2
# ceph orch host add mon3
```



```
[root@mon1 ~]# ceph orch host add osd1
Added host 'osd1' with addr '192.168.30.150'
[root@mon1 ~]# ceph orch host add osd2
Added host 'osd2' with addr '192.168.30.52'
[root@mon1 ~]# ceph orch host add osd3
Added host 'osd3' with addr '192.168.30.209'
[root@mon1 ~]# ceph orch host add mon2
Added host 'mon2' with addr '192.168.30.10'
[root@mon1 ~]# ceph orch host add mon3
Added host 'mon3' with addr '192.168.30.208'
```

Figure 3.43 : Ajout des hôtes au cluster de stockage

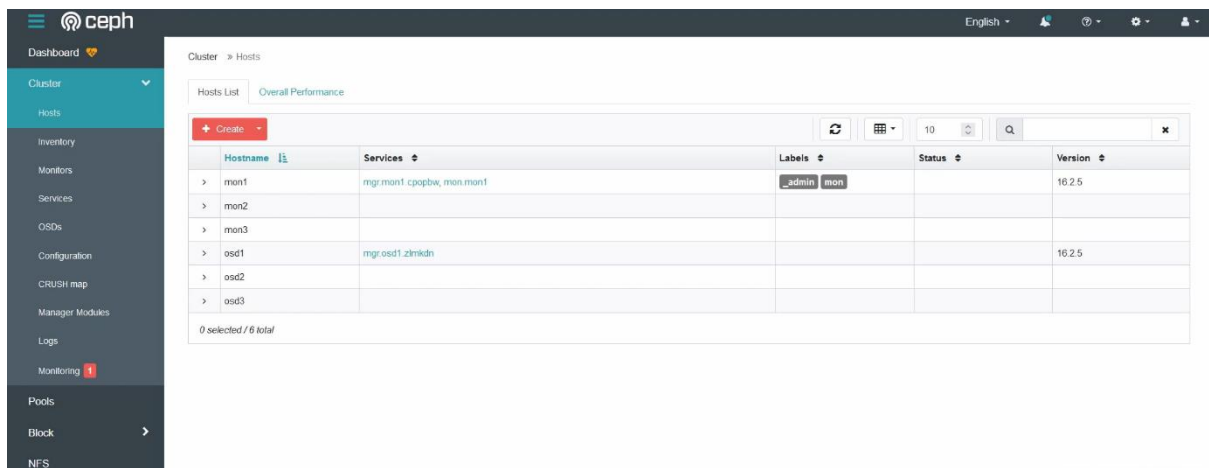


Figure 3.44 : Liste des hôtes après leur ajout

- Lister l'ensemble des serveurs qu'il y a sur la plateforme avec la commande « **# ceph orch host ls** » :

```
[root@mon1 ~]# ceph orch host ls
HOST      ADDR          LABELS        STATUS
mon1      192.168.30.230 _admin mon
mon2      192.168.30.10
mon3      192.168.30.208
osd1      192.168.30.150
osd2      192.168.30.52
osd3      192.168.30.209
```

Figure 3.45 : Liste des hôtes

- ❖ Ajouter les disques disponibles au cluster d'un seul coup sans le faire manuellement l'un après l'autre. Avec le déploiement conteneurisé c'est plus facile et rapide :
 - On affiche les disques disponibles avec la commande « **# ceph orch device ls** » :

```
[root@mon1 ~]# ceph orch device ls
Hostname  Path      Type  Serial          Size  Health  Ident  Fault  Available
osd1      /dev/sda  hdd   drive-scsi0-0-0-0 32.2G Unknown N/A    N/A    Yes
osd1      /dev/sdb  hdd   drive-scsi0-0-0-1 32.2G Unknown N/A    N/A    Yes
osd2      /dev/sda  hdd   drive-scsi0-0-0-0 32.2G Unknown N/A    N/A    Yes
osd2      /dev/sdb  hdd   drive-scsi0-0-0-1 32.2G Unknown N/A    N/A    Yes
osd3      /dev/sda  hdd   drive-scsi0-0-0-0 32.2G Unknown N/A    N/A    Yes
osd3      /dev/sdb  hdd   drive-scsi0-0-0-1 32.2G Unknown N/A    N/A    Yes
[root@mon1 ~]#
```

Figure 3.46 : Liste des appareils disponibles pour déployer les OSDs

- On ajoute les disques disponibles au cluster avec la commande :

```
# ceph orch apply osd --all-available-devices
```

```
[root@mon1 ~]# ceph orch apply osd --all-available-devices
Scheduled osd.all-available-devices update...
[root@mon1 ~]#
```

Figure 3.47 : Consommation de tout périphérique de stockage disponible et inutilisé

ID	Host	Status	Device class	PGs	Size	Usage	Read bytes	Write bytes	Read ops	Write ops
0	osd2	in up	hdd	1	30 GiB	0%	0 B	0 B	0 /s	0 /s
1	osd1	in up	hdd	1	30 GiB	0%	0 B	0 B	0 /s	0 /s
2	osd3	in up	hdd	0	30 GiB	0%	0 B	0 B	0 /s	0 /s
3	osd3	in up	hdd	1	30 GiB	0%	0 B	0 B	0 /s	0 /s
4	osd2	in up	hdd	0	30 GiB	0%	0 B	0 B	0 /s	0 /s
5	osd1	in up	hdd	0	30 GiB	0%	0 B	0 B	0 /s	0 /s

Figure 3.48 : Liste des OSD sur l'interface graphique

- ❖ On en a fini avec les disques. Pour les moniteurs, ils ont été créés cependant ils ne sont toujours pas identifiés comme étant des moniteurs dans le cluster. Pour remédier à ça, on exécute les commandes suivantes :

```
# ssh mon1
```

```
# ceph orch apply mon mon1,mon2,mon3
```

```
[root@mon1 ~]# ceph orch apply mon mon1,mon2,mon3
Scheduled mon update...
[root@mon1 ~]#
```

Figure 3.49 : Déploiement des moniteurs

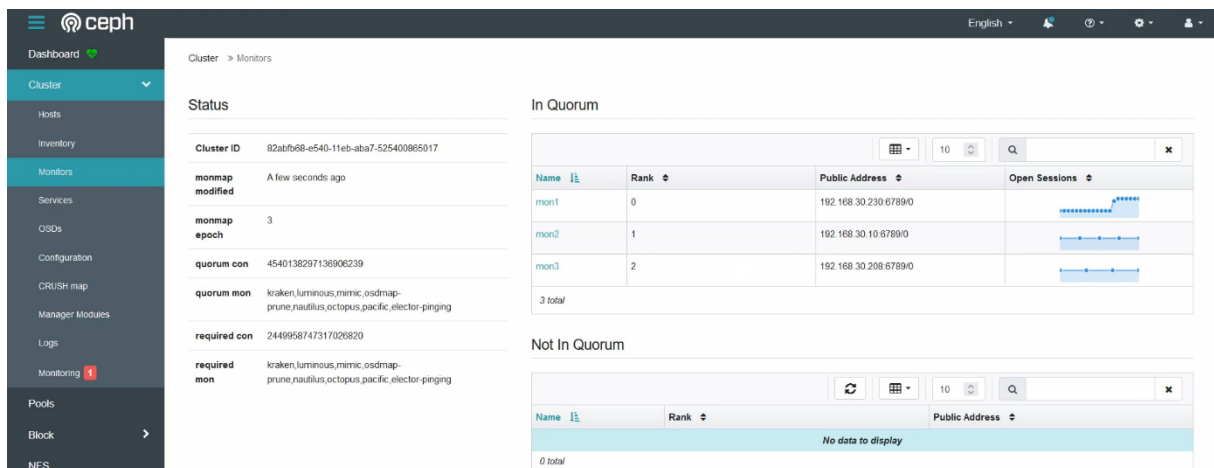


Figure 3.50 : Liste des MON sur l'interface graphique

❖ On a déployé notre cluster de manière conteneurisé avec cephadm.

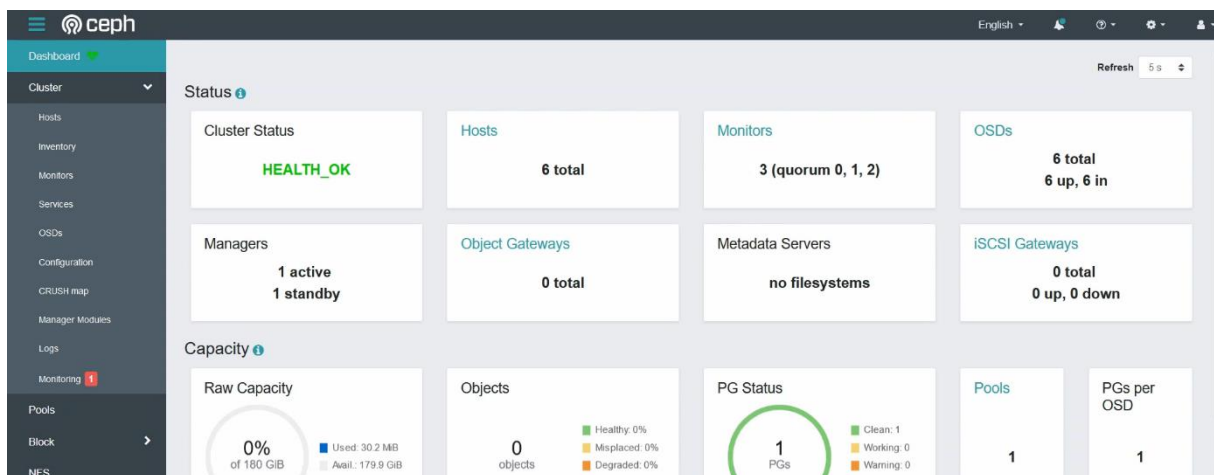


Figure 3.51 : Etat du cluster après sa mise en place

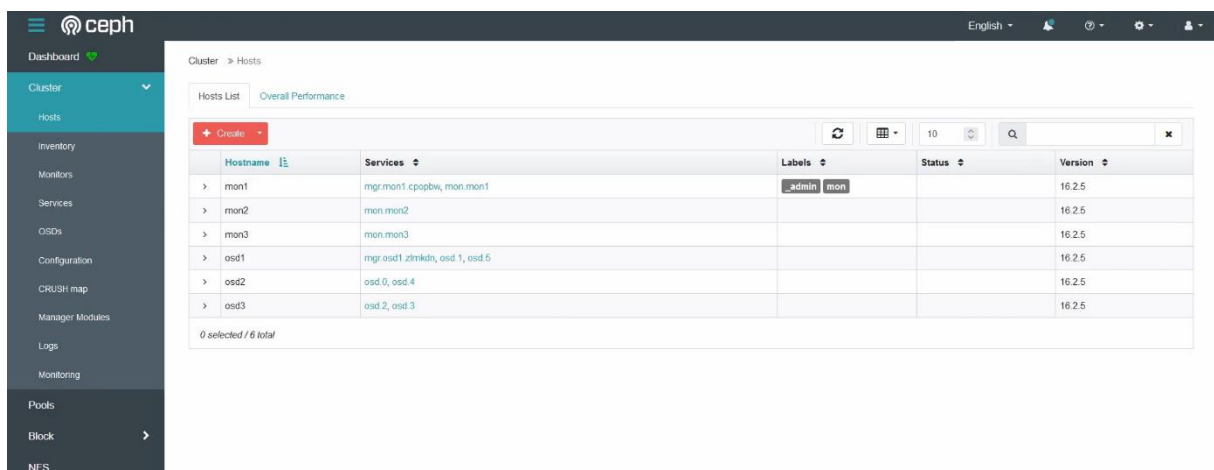


Figure 3.52 : Liste des hôtes et leurs rôles

3.3.5 Création de la Crush Map et stockage des données

Le stockage des données dépend de la politique de stockage qui a été mise en place. Ceph stocke des données dans des pools en premier, puis ces pools seront subdivisés en PG pour qu'ensuite ces PG soient stockés dans les disques. La répartition des objets sur le disque dépend de ce qu'on appelle le « domaine de défaillance » défini par la CRUSH Map. Afin de ne pas perdre les données en cas de panne, il est plus judicieux de créer une crush map de façon à toujours avoir une copie de l'objet même en cas d'arrêt d'un host ou d'un osd ou même d'un rack. Exemple de structuration :

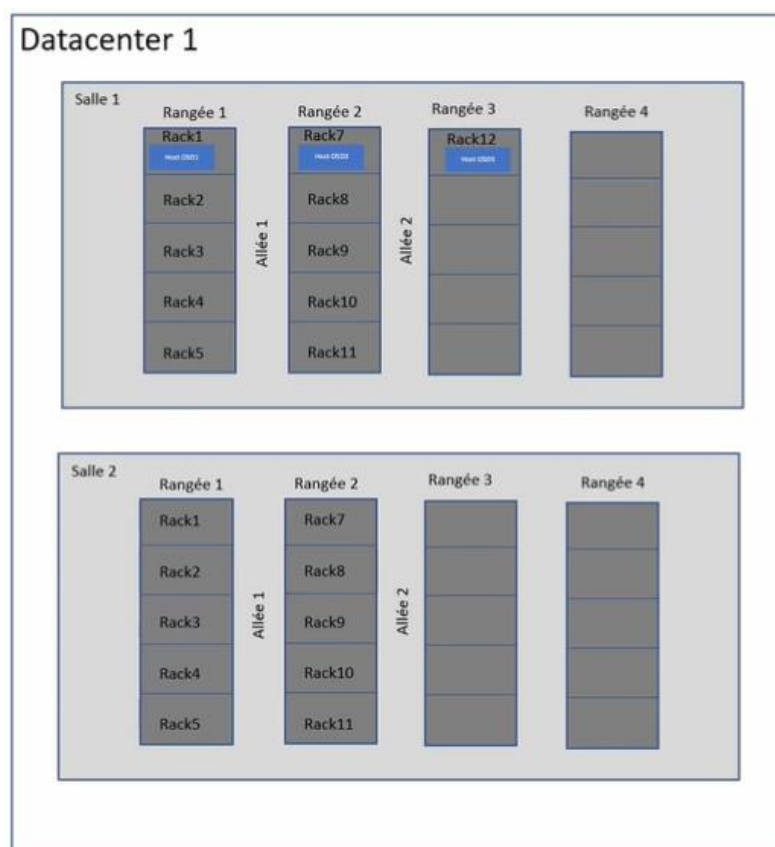


Figure 3.53 : Illustration d'une Datacenter

❖ Définir toutes les variables à utiliser dans la définition de notre Crush map :

```
# ceph osd crush add-bucket dc1 datacenter
# ceph osd crush add-bucket room1 room
# ceph osd crush add-bucket row1 row
# ceph osd crush add-bucket row2 row
# ceph osd crush add-bucket row3 row
# ceph osd crush add-bucket rack1 rack
# ceph osd crush add-bucket rack2 rack
# ceph osd crush add-bucket rack3 rack
# ceph osd crush add-bucket rack7 rack
# ceph osd crush add-bucket rack12 rack
```

❖ Création de la carte :

```
# ceph osd crush move dc1 root=default
# ceph osd crush move room1 datacenter=dc1
# ceph osd crush move row1 room=room1
# ceph osd crush move row2 room=room1
# ceph osd crush move row3 room=room1
# ceph osd crush move rack1 row=row1
# ceph osd crush move rack2 row=row1
# ceph osd crush move rack7 row=row2
# ceph osd crush move rack3 row=row1
# ceph osd crush move rack12 row=row3
# ceph osd crush move osd1 rack=rack1
# ceph osd crush move osd2 rack=rack7
# ceph osd crush move osd3 rack=rack12
```

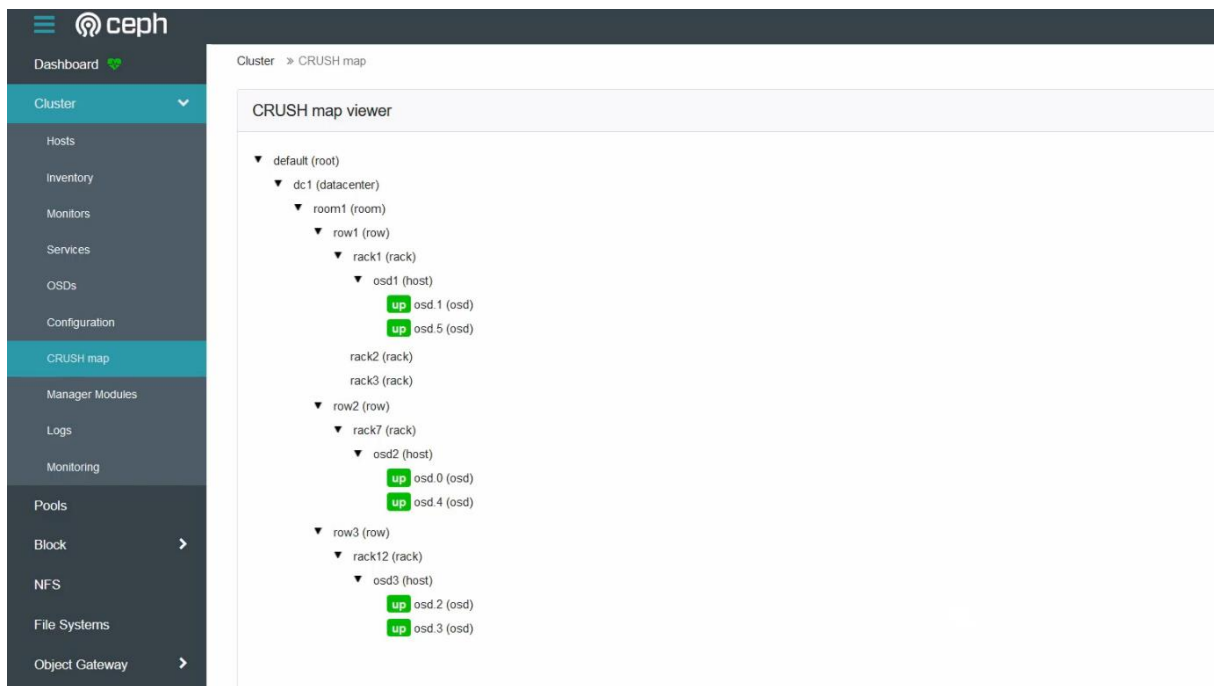


Figure 3.54 : Carte CRUSH

❖ Depuis l'interface web de Ceph :

- Créer le pool : Cliquer sur « Pools » puis « Ajouter » : Nom=pooldata01, type de pool=réplication, placement de groupe=64, taille de la réplication=3, application rbd.

Figure 3.55 : Ajout du pool « pooldata01 »

❖ Depuis l'interface graphique :

- Créer la règle Crush au niveau osd et l'assigner au pool « pooldata01 » :

```
# ceph osd crush rule create-replicated rule_fail_osd1 default osd
```

```
# ceph osd pool set pooldata01 crush_rule rule_fail_osd01
```

```
[root@mon1 ~]# ceph osd crush rule create-replicated rule_fail_osd01 default osd
[root@mon1 ~]# ceph osd pool set pooldata01 crush_rule rule_fail_osd01
set pool 2 crush_rule to rule_fail_osd01
[root@mon1 ~]#
```

Figure 3.56 : Création de la règle CRUSH et l'assigner au pool

- Créer un fichier test qui sera l'objet à stocker :

```
# vi hostsfile.txt
```

- Insérer le fichier créé dans notre infrastructure :

```
# rados -p pooldata01 put object hostsfile.txt
```

- En exécutant la commande « # **ceph -s** » on remarque que l'objet a été ajouté :

```
[root@mon1 ~]# ceph -s
cluster:
  id:      82abfb68-e540-11eb-aba7-525400865017
  health: HEALTH_OK

services:
  mon: 3 daemons, quorum mon1,mon2,mon3 (age 103m)
  mgr: mon1.cpopbw(active, since 13h), standbys: osd1.zlmkdn
  osd: 6 osds: 6 up (since 13h), 6 in (since 13h)

data:
  pools:   2 pools, 65 pgs
  objects: 1 objects, 24 B
  usage:   32 MiB used, 180 GiB / 180 GiB avail
  pgs:     65 active+clean
```

Figure 3.57 : Information sur le cluster

- Lister les objets à l'intérieur du pool « pooldata01 » :

```
# rados -p pooldata01 ls
```

```
[root@mon1 ~]# rados -p pooldata01 ls
object
```

Figure 3.58 : Liste des objets dans le pool « pooldata01 »

- Voir comment cet objet a été réparti à l'intérieur des différents disques :

```
# ceph osd map pooldata01 object -f json-pretty
```

```
[root@mon1 ~]# ceph osd map pooldata01 object -f json-pretty
{
  "epoch": 26,
  "pool": "pooldata01",
  "pool_id": 2,
  "objname": "object",
  "raw_pgid": "2.570e3222",
  "pgid": "2.22",
  "up": [
    5,
    4,
    2
  ],
  "up_primary": 5,
  "acting": [
    5,
    4,
    2
  ],
  "acting_primary": 5
}
[root@mon1 ~]#
```

Figure 3.59 : Emplacement de l'objet créé

3.4 Conception à grande échelle

Sonatrach dispose de cinq raffineries de pétrole à travers le pays : À Sidi R'cine (Alger), à Arzew (Oran), à Skikda, à Hassi-Messaoud (Ouargla), et à Adrar. Leur rôle est de transformer le pétrole en des produits à haute valeur commerciale destinés au marché national et international. Ces raffineries ont permis depuis 2019 d'arrêter l'importation d'essence rendant l'Algérie par la même occasion autosuffisante.

Bien que Sonatrach possède divers domaines d'activités et génère plusieurs données sensibles qui doivent être stockées et traitées, ce chapitre est consacré uniquement sur le stockage de données des raffineries de pétrole.

Dans ce qui suit, on illustre un déploiement de Ceph dans plusieurs points de raffinerie. On présentera notre solution par des schémas et architectures.

3.4.1 Architecture à grande échelle

On propose une solution à grande échelle pour lier les points de raffinage. Il faudra d'abord installer la solution Ceph au niveau des centres de données de chaque raffinerie, puis connecter les clusters entre eux grâce à un VPN.

Les points de raffinerie sont répartis comme dans la figure ci-dessous. Dans chaque centre de données une installation de Ceph est requise :

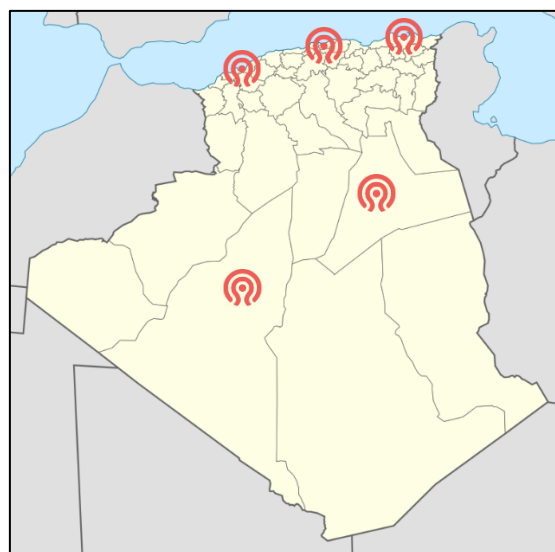


Figure 3.60 : Distribution des clusters Ceph au niveau national

Pour relier les cluster Ceph entre eux, le VPN est la meilleure solution. Il crée un lien entre les différents centres de données permettant une manipulation sécurisée des données.

Le VPN crée un chemin alternatif qui servira de tunneling entre les différents clusters. Il empêchera ainsi toute personne mal intentionnée d'intercepter les informations échangées. En ce qui concerne les données, elles seront chiffrées.

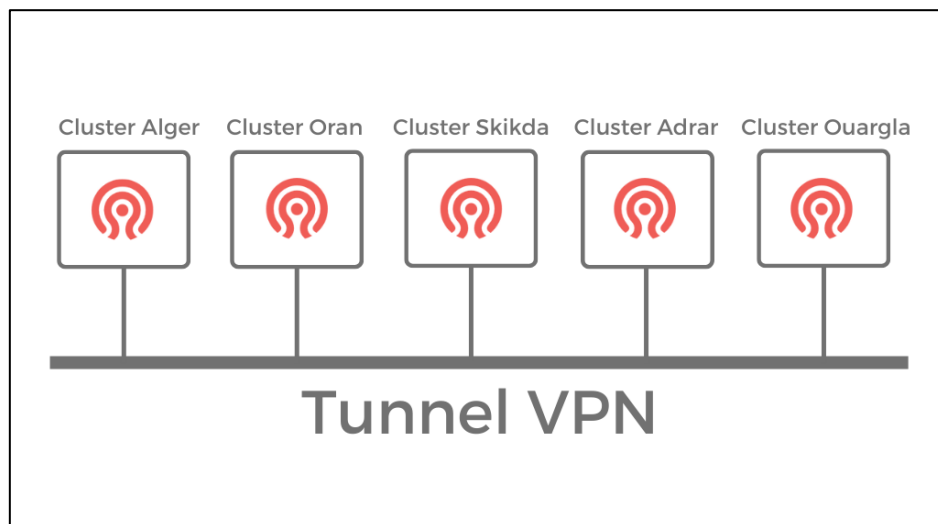


Figure 3.61 : Architecture du déploiement de Ceph

Les clusters de stockage forment une seule et même entité. Il sera possible par la suite de l'étendre en cas d'installation de nouvelles raffineries de façon rapide et simple. La figure ci-dessous représente parfaitement l'architecture dans sa globalité :

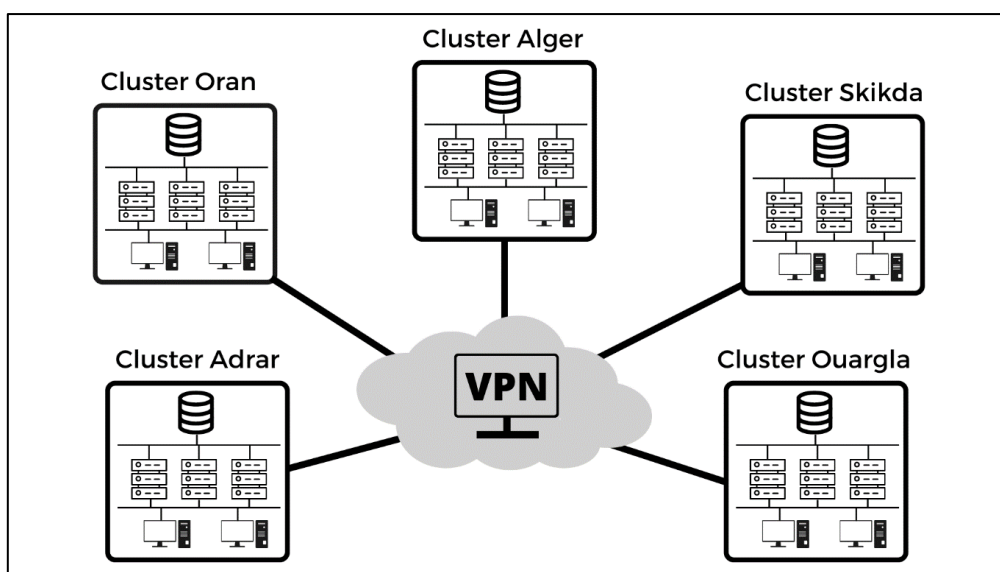


Figure 3.62 : Architecture des clusters Ceph

3.4.2 Spécification des besoins

Pour le déploiement de cette solution il faudra :

- ❖ Au moins deux ingénieurs par raffinerie pour la mise en place des clusters et du réseau.
- ❖ Entre un et trois mois pour la mise en place de Ceph et la migration vers la nouvelle solution de stockage.

3.5 Conclusion

Ce chapitre est l'aboutissement des phases précédentes dans lesquelles le projet a été pensé et décrit. Il nous a permis la mise en place d'un cluster Ceph et l'illustration de la solution Ceph au sein de l'entreprise atteignant ainsi l'objectif du projet.

Conclusion générale

Pour une entreprise qui traite un large volume de données, il est important d'assurer leur disponibilité et leur gestion. La solution de stockage joue donc un rôle primordial pour le bon fonctionnement de l'organisme car les données constituent un élément clé.

Afin de clôturer notre cursus en master informatique, nous avons eu l'opportunité de réaliser un stage pratique au sein d'une grande entreprise nationale ; nous avons abordé la thématique des systèmes de stockage. Notre objectif fut d'étudier et de mettre en place la solution de stockage distribué Ceph. Cela nous a permis de comprendre le grand avantage que cette dernière apportait en termes de stockage, de fiabilité, d'extensibilité et de disponibilité.

A l'issue de la réalisation de ce travail, nous affirmons que notre projet nous a aidé à approfondir nos connaissances en ce qui concerne les systèmes de stockage distribué, en particulier Ceph qui, pour nous, est le futur du stockage.

Ceph offre aux entreprises une meilleure gestion des données. Il facilite leur manipulation et leur traitement, il permet aussi aux employés d'être plus flexible dans leur configuration et être moins stressés en cas de panne. D'autre part, il va permettre d'améliorer la productivité de l'entreprise en termes de temps, diminuer les efforts inutiles et limiter les frais que dépense l'entreprise en termes de maintenance et autres.

Pour conclure, notre travail peut être sujet à des améliorations car Ceph propose une panoplie de configurations offrant plusieurs possibilités de stockage.

Netographie

- [1] <https://www.ummtto.dz/dspace/bitstream/handle/ummtto/2654/m%C3%A9moire%20SONATRACH%281%29.pdf?sequence=1&isAllowed=y>
- [2] <https://www.institut-numerique.org/2-historique-et-evolution-5028f58cb09d3>
- [3] <https://www.zadara.com/blog/2016/04/06/differences-software-defined-storage-traditional-san-nas-cloud-storage-2/>
- [4] https://www.splunk.com/fr_fr/data-insider/what-are-distributed-systems.html
- [5] <https://lucasvidelaine.files.wordpress.com/2020/07/solution-de-stockage-distribuee3a9.pdf>
- [6] https://www.data-transitionnumerique.com/cluster-informatique-grappe-big-data/?fbclid=IwAR2HaBhs5s3zSpJJvAVBBkk_uWh_n_R9hTFfJOmtrCQjFj4WA1V2YEisug
- [7] https://fr.wikipedia.org/wiki/Software-defined_storage
- [8] <https://fr.wikipedia.org/wiki/Ceph>
- [9] <https://documentation.suse.com/fr-fr/ses/5.5/html/ses-all/cha-storage-about.html>
- [10] <https://www.redhat.com/fr/topics/virtualization/what-is-KVM>
- [11] <https://www.commentcamarche.net/telecharger/web-internet/18613-putty/>
- [12] <https://www.udemy.com/course/formation-ceph-mise-en-oeuvre/learn/lecture/26825762#overview>
- [13] <https://docs.ceph.com/en/quincy/>
- [14] https://www.vmware.com/topics/glossary/content/storage-area-network-san.html?fbclid=IwAR1K-BZR8HEdQPXjEfdoveCNtTol_OYQFAipxZpG3je7sJji_-DFutNBlsk
- [15] https://www.inmac-wstore.com/guides-achat-stockage-definition-serveur-nas/cp37621.htm?fbclid=IwAR3Mwm2V0q4FmTm9ezJ5tI7NV_kWrkfYkDooqX6L7TSR1xZBgx2VEBo2Dg0

- [16] <https://sonatrach.com/>
- [17] https://access.redhat.com/documentation/en-us/red_hat_ceph_storage/5/html-single/installation_guide/index
- [18] https://www.lemagit.fr/conseil/Comprendre-les-bases-de-larchitecture-de-Ceph?fbclid=IwAR0OCxbxMZCYxkC_SD8zdWMTFjyVsTd1inmEVW5cMhsgUpDdqyneSEjbhR4
- [19] <https://www.ionos.fr/digitalguide/serveur/know-how/la-solution-ceph/?fbclid=IwAR3MPoCMad4aT5GjpsOQodXh38m2Ea8f3oiXuQxpM2V3UFwk6gHBmt7gctQ>
- [20] https://www.45drives.com/blog/uncategorized/what-is-ceph-why-our-customers-love-it/?fbclid=IwAR3JjqxvqiZJ6VWRTk19S-pgq0Bbwa7dicQvVfifSD_BhCQMLf9zMl63-FU
- [21] https://blog.empreintedigitale.fr/2017/10/23/ceph-technologie-stockage-donnees-open-source/?fbclid=IwAR2eUflyMZNkKBtwSM0M2o_MAs6f6SNrtD4LpJhq2n3FW6dS07PxqamUrA
- [22] <https://medium.com/@dennislee22/all-in-one-ceph-2552f336420e>
- [23] https://www.digitalocean.com/blog/why-we-chose-ceph-to-build-block-storage?fbclid=IwAR1Fnps0W4_qNdXLHIC8A54T4jgvdkJpw7CFNntz_jZS_Y-HkcJPnLTGQX4
- [24] <https://old.ceph.com/geen-categorie/zero-to-hero-guide-for-ceph-cluster-planning/?fbclid=IwAR0RgYi2DeMkAxA1SxQrjZLFeKxNrQ89spIoiuq01im97JIylSpg3NBwwM>
- [25] https://www.journalducsm.com/utilisation-vpn-pour-entreprises/?fbclid=IwAR3R79N_Pd6hL53QZZDIbCYlLfH7xy57HmQlxeWRCW4o5XcjGktF3bHges
- [26] <https://www.supportsages.com/ceph-part-3-technical-architecture-and-components/?fbclid=IwAR28ka-8C3Tqb2eKzkFvliAPtuamC4A7EswUA1Xv1xz-W4kX2JW9niozOmM>
- [27] https://www.netapp.com/fr/data-storage/what-is-san-storage-area-network/?fbclid=IwAR2msxgrKMJn36EDZNqlSuGXj0KcjMaG3n-ttJoH5C90GFUkgxTOSYb_-ws